



DEV&Co

RAPPORT DE SOUTENANCE 3

Objectif Lune !

Objectif Lune !



melody.huang
chloe.eap

hengrui.ye
nael.hulice-mencle

28/05/2025

Table des matières

1	Introduction	2
2	Etapes de la réalisation	3
2.1	Rappel des modifications	3
2.1.1	Suppression des systèmes d'intelligence et de force	3
2.1.2	Changements dans la répartition des tâches	3
2.1.3	Changements dans les comportements des ennemis	4
2.1.4	Changements dans la conception des niveaux	4
2.2	Répartition des tâches	7
2.3	Rappel de la soutenance 1	10
2.3.1	Mélody	10
2.3.2	Hengrui	11
2.3.3	Chloé	12
2.3.4	Nael	13
2.4	Rappel de la soutenance 2	15
2.4.1	Mélody	15
2.4.2	Hengrui	16
2.4.3	Chloé	17
2.4.4	Nael	18
3	Fonctionnalités et déroulé du jeu	19
4	Présentation des tâches qui ont été réalisé	21
4.1	Mélody	21
4.2	Chloé	26
4.3	Hengrui	30
4.4	Nael	34
5	Problèmes rencontrés lors de la réalisation du projet	41
6	Ressenti du projet	45
7	Conclusion	51
8	Source	52
9	Annexes	53

1 Introduction

Pour rappel, “Objectif Lune!” est un jeu de plateforme et d’aventure 2D développé en C# sur Unity. Il est réalisé par l’entreprise Dev&Co, qui est composée de Mélody HUANG, Chloé EAP, Hengrui YE et Nael HULIC-MENCLE en E2.

Le but de ce rapport de soutenance est de montrer notre avancement sur le projet, nos difficultés et la répartition des tâches.

L’histoire d’”Objectif Lune!” se déroule en 2550. Notre héros reçoit alors un message de son père seulement composé d’une position géographique. Curieux, le personnage principal décide de s’y rendre. Son père lui aurait-il préparé une surprise après toutes ces années d’absence ? Arrivé sur place, notre héros n’y trouva rien d’autre qu’une mystérieuse trappe.

“Où peut-elle bien mener ?” N’ayant à peine le temps de se poser cette question qu’une énorme explosion retentit. Pris de peur et de surprise, notre héros se précipite à l’intérieur de la trappe. La nouvelle pièce était plongée dans le noir complet jusqu’à ce qu’une vidéo se lance. C’est une vidéo de la part de son père lui indiquant que, durant ces dernières années, ce dernier était membre d’une équipe de chercheurs dans le nucléaire dont l’expérience ratée a provoqué l’explosion.

L’enregistrement continue son cours en expliquant que les survivants, transformés en mutants, errent à l’extérieur et que notre héros doit maintenant construire une fusée pour s’échapper vers la Lune. Heureusement pour notre personnage principal, le bunker contient tout le nécessaire pour survivre et se préparer : de la nourriture, de quoi se défendre, et bien d’autres choses à découvrir..

2 Etapes de la réalisation

2.1 Rappel des modifications

2.1.1 Suppression des systèmes d'intelligence et de force

Pour mener à bien ce projet, nous avons été obligés d'apporter des modifications sur le jeu tout au long de sa création. En effet, certaines fonctionnalités que nous avons initialement prévues n'auraient pas pu aboutir.

Nous avons donc pris la décision de supprimer les systèmes d'intelligence et de force. Cette suppression a, de fait, entraîné l'élimination des livres ainsi que des mécaniques de synthèse associées. La suppression de la table de synthèse pour la soutenance 1 a entraîné une modification du système de taux d'obtention.

Avant sa suppression nous avons prévu de faire un système qui générerait un certain nombre de matériaux. Dorénavant un niveau donnera un matériel précis alors que certains des niveaux ne donneront rien. Cela évitera au joueur de refaire le même niveau et de ne pas obtenir le matériel nécessaire pour avancer sur le jeu. La suppression du système de force à quant à elle entraîné l'utilisation d'une unique arme, étant la batte.

2.1.2 Changements dans la répartition des tâches

Au début de l'année, nous ne connaissions pas les capacités de chacune des personnes dans le groupe et nous avons donc réparti les tâches à ce qu'elles soient équitables pour tous.

Les mois passèrent, et nous avons remarqué que certains étaient plus à l'aise sur certaines tâches que d'autres. Nous avons donc procédé à un changement des tâches réparties.

Tout d'abord, le code des niveaux : Mélody était au départ la responsable de cette tâche. Cependant, par manque d'inspiration elle a préféré déléguer sa tâche à Chloé qui en est donc devenue la responsable. Chloé s'est occupée du design du menu du jeu et des interfaces, celui de fin de niveau ou encore celui lorsque le personnage meurt. Mélody est alors passée suppléante en s'occupant des différents scripts nécessaires pour réaliser les niveaux. Cette cohésion permet un réel travail de groupe et une bonne communication. Par ailleurs, pour équilibrer les responsabilités, nous avons décidé que Mélody deviendrait responsable du code du bunker et Chloé, sa suppléante.

Toujours au niveau des changements de responsabilités, nous avons énormément eu du mal à comprendre la différence entre le multijoueur et le réseau ainsi que la manière de l'implémenter. Hengrui, ayant pris la grande initiative de se lancer dans ce long parcours pour comprendre et l'implémenter devient le suppléant du réseau en plus d'être le responsable du multijoueur. Quant au suppléant du multijoueur, Chloé s'est désignée pour l'aider.

2.1.3 Changements dans les comportements des ennemis

Concernant les changements dans les comportements des ennemis, nous avons opté pour le remplacement total d'un des comportements. En effet, à la base, les 2 comportements des ennemis étaient ceux du déplacement entre 2 points prédéfinis et du suivi total du personnage principal. C'est ce dernier qui a été remplacé par conséquent pour des problèmes de tags et de scripts, nous avons décidé de le remplacer par un script imitant les mouvements de sauts d'une grenouille permettant une meilleure immersion dans le jeu étant donné le design de certains monstres.

2.1.4 Changements dans la conception des niveaux

Nous avons également apporté des modifications sur les niveaux. Les niveaux ont un rôle crucial sur le jeu. En effet, ils permettent de captiver l'attention pour donner envie au joueur de continuer à jouer.

A l'origine, nous avons défini 12 niveaux avec chacun des thèmes et des objectifs différents. Tout d'abord, nous avons le niveau d'introduction qui permet d'introduire l'objectif et les différentes fonctionnalités du jeu. Ce niveau est très simple avec une petite histoire racontant le synopsis du jeu. Ensuite, 10 niveaux étaient consacrés à la recherche de matériaux pour construire la fusée.

En effet, accomplir un niveau, nous permet d'obtenir des matériaux selon un système de drop que Hengrui avait réalisé. Les défis étaient donc variés, il peut s'agir de tuer tous les ennemis, atteindre la porte de sortie sans se faire toucher par des ennemis, ou encore un labyrinthe d'où il faut s'enfuir. Un défi peut être repris pour un autre niveau mais avec une difficulté en plus, des ennemis plus puissants, ou un champ de vision restreint.

Puis, un niveau secret se débloquent après l'obtention de tous les matériaux et après le premier essai de décollage de la fusée. Une pièce sera manquante, il aurait donc fallu éliminer l'ennemi final. Il s'agissait donc du dernier niveau pour obtenir la pièce manquante.

Difficulté :

- Facile : peu de monstres
- Moyen : plus de monstres
- Difficile : plus de monstres et vision restreinte, c'est-à-dire que le joueur peut voir autour de lui et non au loin.

Explication des différents niveaux :

Niveau	Difficulté	Type de niveau	Cycle	Détails
Niveau 1	difficulté facile	exploration	jour	pas de détails particuliers
Niveau 2	difficulté facile	fuite	jour	les ennemis infligent plus de dégâts
Niveau 3	difficulté moyenne	labyrinthe	jour	il faut trouver la sortie
Niveau 4	difficulté moyenne	exploration	nuit	pas de détails particuliers
Niveau 5	difficulté moyenne	labyrinthe	jour	il faut trouver la sortie en récupérant les pièces cachées de la fusée
Niveau 6	difficulté moyenne	fuite	jour	les ennemis infligent plus de dégâts
Niveau 7	difficulté moyenne	exploration	nuit	pas de détails particuliers
Niveau 8	difficulté difficile	fuite	jour	les ennemis infligent plus de dégâts
Niveau 9	difficulté difficile	labyrinthe	nuit	il faut trouver la sortie.
Niveau 10	difficulté difficile	fuite	jour	les ennemis infligent plus de dégâts
Niveau 11				boss final à éliminer

Finalement, la notion de difficulté ne s'appuie plus seulement sur le nombre d'ennemis, elle s'appuie désormais aussi sur les zones à éviter telles que les trous ou encore de la lave au sol, la taille des plateformes et leurs mouvements ou encore les différents comportements des ennemis. L'option "vision restreinte" a, quant à elle, été supprimée. La répartition des cycles jour/nuit a été gardée mais le contenu des niveaux a été modifié pour certains niveaux. De plus, leurs conceptions seront ici plus développées puisqu'il ne s'agit plus d'idées à développer mais d'idées développées.

Niveau 1 :

Etat : inchangé

Détails : Niveau accessible à tous types de joueurs : ennemis de type "limace", peu de trous, plateformes grandes et faciles d'accès

Niveau 2 :

Etat : inchangé

Détails : Introduction de l'ennemi de type "grenouille" en plus des limaces. Premier niveau de fuite : les ennemis infligent trois fois plus de dégâts. Niveau restant accessible à tous.

Niveau 3 :

Etat : modification vers un niveau de type énigme, sous forme de questions à choix multiples

Détails : Introduction d'un nouveau personnage : Arachnée, la reine des araignées.

Niveau 4 :

Etat : inchangé

Détails : Premier niveau de nuit, introduction d'un nouveau type d'ennemi : la chenille.

Niveau 5 :

Etat : modification vers niveau de type énigme

Détails : Retour d'Arachnée avec des questions plus compliquées qu'au troisième niveau.

Niveau 6 :

Etat : inchangé

Détails : Premières plateformes mouvantes. Forte présence de grenouilles : niveau créé pour tester les capacités d'observation et de saut du joueur.

Niveau 7 :

Etat : inchangé

Détails : Introduction d'un nouveau type d'ennemi : la fourmi.

Niveau 8 :

Etat : modification vers un niveau d'exploration, de labyrinthe

Détails : Niveau sans ennemi, le but est de choisir le bon chemin afin d'atteindre la fin du niveau. Attention aux murs invisibles !

Niveau 9 :

Etat : modification vers un niveau de fuite où les ennemis infligent trois fois plus de dégâts

Détails : Attention aux fourmis volantes

Niveau 10 :

Etat : inchangé

Détails : Le sol est majoritairement composé de lave, nécessite d'utiliser les plateformes mouvantes

Niveau 11 :

Etat : inchangé

Détails : Changement de décor vers un laboratoire abandonné. Niveau sans ennemi excepté le boss final situé dans la chambre du boss, à la fin du niveau.

2.2 Répartition des tâches

Dans le but d'obtenir un jeu jouable à la fin de l'année, nous avons réparti les différentes tâches dans le groupe.

Voici le tableau des répartitions des différentes tâches afin de réaliser le jeu vidéo. Chaque tâche dispose d'un responsable et d'un suppléant qui aidera et complètera la tâche.

Ce que nous avons initialement prévu :

Noms/Tâches	Mélody HUANG	Chloé EAP	Hengrui YE	Nael HULICE-MENCLE
Graphisme et Jaquette	Suppléant	Responsable		
Base du jeu	Suppléant		Responsable	
Son			Suppléant	Responsable
Code du personnage			Responsable	Suppléant
Code des niveaux	Responsable	Suppléant		
Code du bunker		Responsable		Suppléant
Site	Responsable			Suppléant
Intelligence Artificielle			Suppléant	Responsable
Multijoueur		Suppléant	Responsable	
Réseau	Responsable		Suppléant	

Après modifications :

Noms/Tâches	Mélody HUANG	Chloé EAP	Hengrui YE	Nael HULICE-MENCLE
Graphisme et Jaquette	Suppléant	Responsable		
Base du jeu	Suppléant		Responsable	
Son			Suppléant	Responsable
Code du personnage			Responsable	Suppléant
Code des niveaux	Suppléant	Responsable		
Code du bunker	Responsable	Suppléant		
Site	Responsable			Suppléant
Intelligence Artificielle			Suppléant	Responsable
Multijoueur		Suppléant	Responsable	
Réseau	Responsable		Suppléant	

Tout d’abord, le code du personnage joue un rôle crucial dans le jeu. Cette tâche concerne le développement du personnage principal que le joueur incarnera tout au long de son aventure. Le code du personnage implique la programmation de ses différents mouvements, tels que le saut, le déplacement et l’attaque. Sans ce personnage, aucune interaction significative ne peut avoir lieu, rendant ainsi impossible l’engagement du joueur dans l’univers du jeu. Cette tâche importante a été confiée à Hengrui, qui a été assisté par Nael, garantissant ainsi une réalisation soignée et efficace de toutes les mécaniques de jeu liées au personnage.

Deuxièmement, le graphisme joue un rôle fondamental en offrant un visuel attrayant et immersif au jeu. Les différents éléments graphiques ont été dessinés et réalisés par Coline EAP, une étudiante talentueuse en école d’animation. Elle suit attentivement nos précisions et nos attentes concernant les exigences esthétiques que nous avons fixées. Chloé se charge alors de diriger le processus de création des graphismes dans les moindres détails, bénéficiant de l’aide précieuse de Mélody.

Troisièmement, le son joue un rôle essentiel dans l’immersion au sein du jeu. En effet, divers effets sonores sont utilisés pour plonger le joueur dans un environnement unique et captivant. Chaque événement du jeu est associé à un son spécifique : par exemple, un son distinct pour les combats et un autre pour l’ambiance du bunker. Ces différents sons sont soigneusement réalisés par Alexandra TOURE, une passionnée de musique, qui suit les directives de Nael tout en bénéficiant de l’aide précieuse de Hengrui. Grâce à cette collaboration, l’expérience sonore enrichit véritablement le déroulement du jeu, rendant chaque moment encore plus engageant et mémorable.

Par ailleurs, un jeu ne peut pas se réaliser sans défis. Nous avons donc décidé d’implémenter différents niveaux avec des difficultés et des objectifs différents pour ajouter du challenge au jeu. Cette tâche est réalisée par Chloé aidée par Mélody.

Notre jeu se déroule dans un monde apocalyptique, où la présence d’un bunker revêt une importance stratégique pour “Objectif Lune!”. En effet, il ne s’agit pas d’un simple bunker, mais d’un point fondamental qui possède différentes fonctionnalités essentielles. Ce lieu abrite un ordinateur destiné aux tutoriels disponibles pour le joueur, ainsi qu’une carte du monde permettant la sélection des niveaux. De plus, c’est également l’endroit où la fusée devra être construite, ajoutant une dimension cruciale à l’aventure. Le design et la réalisation du bunker ont été confiés à Mélody, qui a bénéficié de l’aide précieuse de Nael.

Par la suite, nous avons le site internet, qui joue un rôle clé en rendant notre jeu accessible à tous. Ce site regroupe non seulement les avancements du projet, mais aussi des détails

essentiels concernant le jeu, permettant aux visiteurs de suivre notre progression. Réalisé en HTML, CSS et JavaScript, le site a été conçu par Mélody, qui a bénéficié du soutien de Nael.

Ensuite, nous avons les bases du jeu qui regroupent la distribution des différents matériaux. En effet, chaque matériau pourra être obtenu à la fin d'un niveau si l'objectif est atteint. Ces bases sont réalisées par Hengrui avec le soutien de Mélody.

De plus, l'intelligence artificielle est essentielle pour rendre le jeu plus automatique et fluide. Chaque ennemi se déplacera en fonction du contexte. Par exemple, un ennemi se déplacera uniquement là où cela est possible. Cette programmation est gérée par Nael, avec le soutien de Hengrui.

Le jeu sera disponible en mode solo, mais également en multijoueur. L'importance du mode multijoueur réside dans la possibilité de réaliser un niveau à plusieurs, ce qui favorise non seulement des interactions enrichissantes avec d'autres joueurs, mais aussi la création de liens significatifs. Ce mode nécessite une communication efficace et un véritable travail en équipe pour surmonter les défis ensemble. La mise en place de cette fonctionnalité est assurée par Hengrui, qui travaille en étroite collaboration avec le soutien de Nael.

Et enfin, le réseau, il a pour but de permettre le multijoueur sur un même réseau internet. Le réseau est réalisé par Mélody, aidée par Hengrui.

2.3 Rappel de la soutenance 1

2.3.1 Mélody

A la première soutenance, je m'étais occupée principalement de la réalisation des niveaux des premières idées que nous avions eu.

Pour réaliser ces niveaux, j'avais d'abord développé une scène simple d'où le personnage principal se téléporte automatiquement vers le bunker lorsqu'il arrivera à la sortie du niveau. Une fenêtre apparaissait également pour afficher les matériaux obtenus si le joueur avait réussi le niveau. Sinon, une autre s'affichait demandant si le joueur souhaitait retourner dans le bunker ou recommencer le niveau. Je m'étais donc également concentrée sur la partie fonctionnelle des niveaux et non du visuel. En effet, les graphismes n'étant pas finis, nous avons pris un personnage vert sur Internet, libre de droit. Ce personnage représentait l'emplacement des ennemis qui sera remplacé une fois les graphismes finis. Le personnage principal était situé tout à gauche au début des niveaux. De plus, la statue représentait la sortie du niveau. Le personnage ne pourrait sortir qu'une fois l'objectif atteint.

La difficulté majeure lors de la création des différents niveaux était le changement de niveau. J'avais eu du mal à comprendre Unity et son fonctionnement. J'avais donc pour l'instant créer différents niveaux qui existaient. Pour résoudre le problème de changement de niveau, j'avais suivi différents tutoriels et essayé d'implémenter. Pour le moment, le changement de niveau n'était pas fonctionnel et la création du choix, une fois le niveau terminé.

J'avais également implémenté un visuel lorsque le joueur mourra. Le joueur avait alors le choix de recommencer le niveau ou de retourner au bunker. Cette fonctionnalité n'était pas parfaite. En effet, lorsque nous lançons ce niveau, la fenêtre s'ouvrait automatiquement sans pouvoir faire autre chose. Pour faire face à ce problème, j'avais décidé de créer une autre scène et de la faire superposer par-dessus la scène. Cette solution n'était pas évidente et risquait de générer des erreurs.

Ensuite, j'ai commencé à développer le site internet. Pour le moment, il n'était pas hébergé, mais il sera prochainement accessible via un lien. Le site a été développé en HTML et en CSS. J'ai eu la chance d'avoir eu auparavant des stages qui m'ont initiée en HTML et en CSS. Malgré tout, je ne me rappelais plus exactement leurs fonctionnements. J'avais donc suivi un tuto pour débiter puis j'avais fait quelques recherches sur Internet pour trouver comment faire certaines fonctionnalités.

Quand nous passons la souris sur l'une des catégories en haut, celle-ci se blanchissait. De plus, lorsque nous passons d'une page à l'autre, le nom concerné se blanchissait et son

écriture devenait noire. Elle permettait d'indiquer sur quelle page est actuellement l'utilisateur. Cependant, le contenu des autres pages n'avait pas encore été réalisé.

Étant responsable du réseau, je n'avais, pour l'instant, pas touché à cette partie. En effet, dans notre groupe, nous n'arrivions pas à faire la différence entre multijoueur et réseau qui était identique pour nous. Cependant, nous avons seulement réussi à faire la différence après que Hengrui ait commencé le multijoueur sans se rendre compte qu'il faisait le réseau. C'est pour cela que Hengrui avait donc commencé le réseau, qui sera expliqué dans sa partie.

2.3.2 Hengrui

Pour la première soutenance, je m'étais occupé du code du personnage, notamment les mouvements les plus basiques, c'est-à-dire les mouvements horizontaux (de gauche à droite), mais aussi les sauts. Durant la réalisation de ces mouvements, le saut fut un mouvement plus compliqué que les deux autres. C'est-à-dire que les mouvements horizontaux n'avaient aucune contrainte difficile : il suffit que le personnage puisse se déplacer lorsqu'on appuie sur la touche dédiée, alors que le saut possède une contrainte qui le rend plus complexe : le saut doit pouvoir être réalisé seulement si le personnage principal est au sol. Et pour réaliser ceci, je me suis servi d'un composant de la classe `GameObject` qui est le composant `tag`.

L'ajout de la fonction `"OnCollisionEnter2D"` est l'élément crucial faire sauter le personnage principal seulement lorsqu'il est au sol. Ce fonctionnement permet de transformer un booléen `"grounded"` seulement lorsque le personnage principal est en contact avec un élément qui possède le tag `"Ground"`. De ce fait, si on attribue le tag `"Ground"` au sol, alors le personnage principal ne pourra sauter que s'il est en contact avec le sol.

Je m'étais aussi renseigné à propos du réseau, et commencé la réalisation de celui-ci. Avec l'aide du `Netcode for GameObject`, un système proposé par Unity, la réalisation du début pour le réseau fût sans grosse difficulté. C'est-à-dire qu'à l'aide du `Netcode for GameObject` de nombreuses tâches sont directement réalisées par Unity, comme la synchronisation des données entre plusieurs joueurs.

Pour l'instant, le jeu peut fonctionner sur plusieurs personnages sur différentes fenêtres, cependant les différentes fenêtres doivent être lancées sur un même ordinateur.

Ceci est un code simple qui permet au joueur d'accéder au `"host"`, ou au client ou au serveur lorsqu'il appuie sur l'un des trois boutons. Pour l'instant, il ne sert seulement qu'à

tester le fonctionnement du jeu sur différentes fenêtres, mais ce type de code va aussi beaucoup aider par la suite, notamment pour les boutons de choix du mode solo ou multijoueur.

Pendant la réalisation du réseau, j'ai aussi rencontré quelques problèmes, qui ont pu être résolus à l'aide de quelques composants qui sont déjà présents dans Unity.

Le premier problème que j'ai rencontré est le déplacement du personnage, c'est-à-dire que lorsqu'on cherche à déplacer un personnage, l'autre personnage se déplace aussi. Ce problème est dû au fait que dans le script du mouvement, on n'a pas différencié les joueurs. Et donc pour résoudre ce problème, j'ai trouvé une propriété qui est déjà présente dans Unity.Netcode qui est le booléen "IsOwner", qui permet de dire si l'action demandée provient du joueur local.

Le deuxième problème est la synchronisation des données. C'est-à-dire que lorsqu'un des personnages se déplace sur l'une des 2 fenêtres, sur l'autre fenêtre, le mouvement n'est pas abouti, ce qui pose donc un problème majeur. Cependant, grâce à un autre composant de Unity, qui est le Client NetWork Transform, ce problème a été résolu facilement. Car avec l'ajout de ce composant dans le préfabriqué du joueur, les données de l'action sont directement envoyées à l'autre client, ce qui permet donc la synchronisation des données et donc des mouvements.

2.3.3 Chloé

Au niveau des graphismes, nous avons terminé une grande majorité des designs comprenant celui du personnage principal, de l'ensemble du bunker, et des niveaux. Autrement dit, nous nous étions mis d'accord sur ce que devait dessiner Coline et nous savions à quoi devait ressembler notre jeu visuellement, la jaquette ainsi que l'animation du premier lancement du jeu. (voir annexe)

Le personnage principal porte une tenue civile puisque l'histoire se déroule en 2550 et qu'il a été pris au dépourvu par l'explosion. Les couleurs de ses habits sont plutôt sombres pour représenter le côté effacé et fatigué du personnage principal, mais aussi pour qu'il puisse se fondre dans le décor du jeu et ainsi éviter de se faire repérer par les ennemis. Les couleurs principales du jeu étaient le rouge pour représenter le danger et le vert pour contraster avec le rouge.

Les mutants ont été dessinés avec des éléments humains, mais aussi des éléments d'animaux pour représenter la nature punissant l'homme après son échec, l'explosion nucléaire.

Le mutant papillon de nuit est le boss final. Celui-ci possédait deux formes, la première invulnérable, la deuxième dévoilant ses points faibles : ses yeux.

Le bunker a été pensé pour être le plus minimaliste possible, il n'est censé contenir que le nécessaire à sa survie mais aussi le nécessaire pour les fonctionnalités principales du jeu. La carte du monde a, quant à elle, été pensée pour imiter une enquête menée par le personnage principal. Elle évoluera en fonction de l'avancée du joueur en dévoilant petit à petit les différents niveaux. (voir annexe)

La jaquette du jeu reprend l'une des couleurs principales du jeu : le rouge. Nous y verrons le personnage principal dos à nous, face à la lune et équipé de son arme fétiche, une batte, entourée d'une ville en ruine. (voir annexe) Comme le suggère l'illustration, nous avons prévu d'écrire le titre du jeu dans l'ombre du personnage principal.

Toujours au niveau du graphisme, il nous restait donc à réfléchir au déroulement de l'animation qui devait clôturer le jeu, c'est-à-dire le décollage de la fusée, ainsi que l'apparence des composants de la fusée et celui de la fusée en elle-même. Les plateformes qui constituent les niveaux ont été reprises d'un site en libre de droit.

Au niveau du bunker, ne possédant pas encore les dessins officiels du bunker, j'avais donc créé un prototype de bunker à partir d'images auxquelles j'avais attaché du code. J'avais donc commencé le code de l'inventaire et du point de sauvegarde sans pour autant les avoir finalisés. Théoriquement, il n'était pas difficile de les programmer. La difficulté principale résidait dans la synchronisation avec l'interface graphique : j'ai eu du mal à trouver comment faire interagir le personnage principal avec les éléments de même pour comment faire apparaître les différentes fenêtres après l'interaction.

2.3.4 Nael

Pour détailler ce qui a été fait pour la première soutenance, je m'étais principalement documenté sur les outils que j'aurais pu utiliser afin d'implémenter efficacement une IA dans leur jeu.

Pour cela, plusieurs méthodes peuvent être mises en œuvre, mais j'ai préféré opter pour l'utilisation de capteurs et de calcul de distance entre l'ennemi et le personnage. La partie de code faite à cette date a donc été centrée sur le suivi du joueur durant les niveaux ainsi que les attaques que peuvent subir le joueur. Ce qui a été le plus dur concernant la partie sur l'IA a été de comprendre ce qu'il fallait réellement faire.

Par la suite, cela a été de réussir à trier les idées et la réalisation de celle-ci. Cela a été plutôt fastidieux et a pris du temps d'où le peu de codes qui avaient été créés. Cependant, à force de se documenter sur le sujet, plusieurs idées sont devenues envisageables.

Pour la partie des bandes sonores, l'aide d'Alexandra, une passionnée de la musique a été indispensable. L'utilisation première des bandes sonores dans notre jeu a donc été tournée vers les 4 moments suivants : la première a été pour l'écran d'accueil, la seconde était pour le menu de sélection de niveaux ainsi durant l'ouverture de la carte, la troisième était initialement pour l'intérieur du bunker, la quatrième bande sonore ainsi que toutes ses déclinaisons étaient destinées aux niveaux.

Afin de diversifier les émotions véhiculées par les musiques, celles-ci ont été créées spécialement pour leur jeu. Les idées principales venaient des couleurs, le jeu des couleurs étant assez important pour le choix des morceaux à implémenter dans le jeu. Étant donné l'environnement apocalyptique du jeu, les musiques qui ont été présentées durant le déroulement du jeu ne peuvent pas être joyeuses. C'est pourquoi la plupart des bandes sonores sont plutôt calmes et font appel à un environnement en ruine. Le choix des musiques est important dans un jeu pour transmettre des émotions différentes selon le moment.

En termes d'inspiration, des inspirations variées et diversifiées ont été choisies, provenant tout d'abord d'inspiration de musiques d'animés (japonaises), par la suite l'ambiance de Geometry Dash et Undertale, étant un jeu sorti en septembre 2015, ont aussi été des sources d'inspiration. Le style de musique principalement présent dans le jeu est du type électronique. À ce moment-là, plusieurs bandes sonores étaient quasiment prêtes, mais méritaient tout de même quelques retouches afin d'être finalisées. Il était donc prévu de finir ces bandes sonores afin d'avoir le temps de les implémenter dans le jeu vidéo. Ainsi, plusieurs variations de bandes sonores ont été en production afin de varier les émotions du joueur.

2.4 Rappel de la soutenance 2

2.4.1 Mélody

Pour cette seconde soutenance, j'avais pu avancer sur le site internet. De nombreuses informations avaient pu être ajoutées sur le site tels que le téléchargement des rapports précédents, le synopsis du jeu, les sources, les descriptions des membres du groupe. De plus, pour ajouter plus de dynamisme sur le site, j'ai ajouté du JavaScript qui a permis, par exemple, d'avoir un calendrier où au clic sur un mois permettait d'afficher ce qui avait été réalisé sur le projet.

De plus, j'avais contribué à la réalisation du bunker, le lieu qui regroupait toutes les informations nécessaires pour jouer telles que l'inventaire ou encore la carte du monde. Pour la réalisation de l'inventaire et de la carte j'avais utilisé des panels : il s'agit d'interfaces qui apparaissent à l'écran.

Ces interfaces s'affichaient uniquement lorsque le joueur était proche et qu'il appuyait sur E pour l'inventaire et T pour la carte. Un texte apparaissait également en bas à gauche pour prévenir le joueur qu'il était possible d'interagir avec un objet. Cette interaction avait pu être réalisée à l'aide des box collider 2D que propose Unity. En effet, avec la fonctionnalité is trigger, elle permettait au joueur de se superposer à l'objet tandis que le code permettait de vérifier si le joueur actuel était proche à l'objet grâce aux tags qui avaient été attribués.

Les scripts pour les affichages avaient été réalisés sans difficulté. En effet, la première était un peu compliquée, notamment pour l'inventaire puisque j'avais encore du mal à me familiariser. J'avais donc suivi de nombreux et divers tutoriels, ce qui m'avait permis de le réaliser avec succès. J'avais par la suite pu l'implémenter pour la carte sans souci majeur puisque les étapes étaient exactement identiques.

Le système de carte avait été réalisé en faisant en sorte que le niveau suivant ne soit pas réalisable si le niveau précédent n'avait pas été réussi. Cette fonctionnalité n'était pas encore très au point puisque le système de sauvegarde n'avait pas été réussi. J'avais tenté d'implémenter un premier essai, mais sans succès.

J'avais pu également améliorer le système de changement de niveau. A la soutenance 1, le changement de niveau était instantané lorsque l'on arrivait à un certain point. Maintenant, il fallait interagir avec une statue pour terminer le niveau. Une interface s'affichait demandant au joueur s'il souhaitait passer au niveau suivant ou retourner au bunker.

Par ailleurs, n'étant pas très talentueuse au niveau des réalisations graphiques et n'ayant pas d'imagination, Chloé s'était occupée du design du menu du jeu, des interfaces de fin de niveau ou encore lorsque le personnage mourrait. Je m'étais donc chargée seulement du script. Cette cohésion permettait un réel travail de groupe et une bonne communication. Sans elle, le jeu n'attirait pas les futurs joueurs.

2.4.2 Hengrui

Pour cette deuxième soutenance, j'ai tout d'abord essayé de terminer le code du personnage, en implémentant la barre de vie du personnage, les animations des mouvements et la caméra du personnage. Durant la réalisation de ces différentes tâches, je n'ai fait face à aucune réelle difficulté.

Pour la barre de vie, j'ai seulement eu besoin d'implémenter plusieurs canvas et images dans le projet, ainsi qu'un code C# permettant de remplir la barre de vie et de gérer la couleur en fonction du nombre de points de vie restants.

J'avais utilisé 3 variables, le Slider qui représente la barre de vie, le gradient un dégradé de couleur qui varie en fonction de la vie, et le fill qui est l'image qui remplit la barre de vie. Une première fonction permet de remplir la barre de vie lorsque le nombre de points de vie est au maximum, et une deuxième fonction permet donc de faire varier la barre de vie, lorsque le personnage principal prend des dégâts.

L'animation du personnage a pu être réalisée grâce à l'ajout d'un composant nommé Animator dans le script du personnage principal. Pour la réalisation de cette animation, j'ai simplement superposé les séquences d'images fournies par Coline sur un intervalle de temps. Afin que cette animation ne fonctionne uniquement lorsque le personnage est en mouvement, j'ai créé deux animations : la première représente le personnage à l'arrêt et la deuxième s'active lorsqu'il se déplace. Pour relier ces animations, j'ai ajouté dans le code C# du déplacement un script affiché ci-dessous, permettant d'obtenir le composant Animator, puis de l'appeler pour déclencher les animations appropriées.

La seule difficulté que j'ai rencontrée pour l'animation de la marche a été le fait que, lorsque l'animation s'active, le personnage devient plus grand qu'avant. Ce problème était causé par le fait que les images utilisées pour le personnage et l'animation de la marche n'étaient pas de la même taille. Ce problème a été résolu assez facilement en redimensionnant l'image d'origine.

Pour la caméra du personnage, j'avais au départ choisi d'utiliser un système proposé par Unity, le Cinemachine Caméra, qui fonctionne parfaitement en solo. Cependant, lorsqu'on passe au mode multijoueur, Cinemachine Camera ne permet plus d'assurer le suivi de la caméra. Avec l'utilisation de Cinemachine Camera, ce suivi n'était effectué que sur le dernier personnage apparu dans la scène du jeu, ce qui ne correspondait pas à notre objectif : nous voulions que la caméra suive chaque personnage individuellement.

Aucune solution n'a été trouvée car la plupart des tutoriels disponibles dataient d'anciennes versions d'Unity et ne fonctionnaient plus avec notre version. C'est pourquoi j'ai décidé d'adopter une autre approche en réalisant ce suivi de caméra avec un script en C#. Grâce à ce code, le suivi de caméra est désormais opérationnel en mode multijoueur, avec la possibilité de définir des limites de position empêchant la caméra de se déplacer au-delà de certaines zones. La mise en place du mode multijoueur n'a donc pas été une grande difficulté puisque comme mentionné précédemment, le seul problème rencontré a concerné le suivi de la caméra.

Mais aussi la gestion des animations en mode multijoueur. Grâce à un composant fourni par NetCode for GameObjects d'Unity, le Network Animator, il a suffi d'ajouter ce composant au script de notre personnage pour que l'animation fonctionne parfaitement en mode multijoueur.

2.4.3 Chloé

Ma tâche principale a été de synchroniser toutes les parties du jeu que l'on possédait en un seul projet Unity. C'était une mission plus longue que ce que l'on pourrait croire car il a fallu refaire tous les aspects visuels de chaque partie récupérée : refaire l'animation du personnage principal lors de son importation, reconnecter toutes les fonctions à leurs panels et boutons respectifs sans oublier tous les problèmes liés à régler et les fichiers à télécharger pour le multijoueur.

En plus de cette mission, j'étais toujours chargée des graphismes en nous mettant en lien avec notre illustratrice Coline. Pour ce côté-là, les graphismes liés aux niveaux et au bunker étaient tous totalement finis.

En ce qui concerne les niveaux, en tant que suppléante et comme dit dans la partie de Mélody, je me suis occupée de dessiner les niveaux à partir des dessins montrés plus tôt ainsi que de plateformes prises d'Internet, en libre-service. Globalement, je me suis donc occupée du visuel des niveaux et, de la même manière que pour la fusion de l'ensemble du jeu en un projet, de la coordination des codes de Mélody avec ma partie visuelle. Tous les niveaux

devaient être dessinés à partir de ces ressources en dehors des niveaux 3 et 5 puisqu'ils ne sont pas des niveaux d'exploration. Ce sont des niveaux "énigme", des niveaux dans lesquels le joueur devra utiliser ses connaissances pour les passer. Ils se déroulent sous forme de QCM.

Tandis que les niveaux 1, 2 et 10 sont des niveaux de jeux de plateformes classiques.

Les capsules des aperçus des niveaux 2 et 10 représentaient en réalité des ennemis que je n'avais pas encore eu le temps de mettre, les jaunes correspondaient au suivi du personnage(s) tandis que les blancs concernaient le comportement de ceux qui faisaient des allers-retours.

2.4.4 Nael

Tout d'abord concernant la partie sonore, un important travail d'écoute et de sélection minutieuse des sons et bandes sonores appropriées a été nécessaire afin de les adapter aux différents moments et ambiances du jeu.

Cette phase de sélection et de peaufinage de l'environnement sonore a représenté la majeure partie des efforts consacrés à cet aspect du projet. En effet, il a fallu passer au crible de nombreuses pistes audio, en testant leurs adéquations avec les différentes scènes et situations du jeu, avant de retenir celles qui seraient finalement implémentées.

Par la suite, l'intégration technique de ces bandes sonores sélectionnées s'est avérée relativement aisée, grâce à l'utilisation d'un script C# spécifiquement développé pour permettre la lecture des fichiers audio fournis en amont. Afin d'assurer une parfaite cohérence entre l'environnement sonore et les animations visuelles, l'option de lecture en boucle (loop) intégrée au moteur de jeu Unity a donc été utilisée. Cette fonctionnalité permet aux fichiers audio d'être lu en continu sur l'ensemble des niveaux et scènes du jeu, garantissant ainsi une immersion sonore constante et fluide.

En ce qui concerne les choix d'implémentation des sons, il a été décidé de les intégrer à différents niveaux : dans les scènes de jeu elles-mêmes, dans la base principale du joueur, ainsi que dans l'écran d'accueil du jeu. Cette répartition a permis de créer un environnement sonore riche et varié, apportant une dimension supplémentaire à l'expérience de jeu.

Concernant maintenant l'implémentation de l'intelligence artificielle, celle-ci repose sur l'automatisation des mouvements et comportements des ennemis rencontrés par le joueur.

Pour ce faire, une multitude de scripts C# ont été créés et testés, représentant plus de 8 versions différentes, avant d'en retenir seulement deux répondant parfaitement aux objectifs fixés. Il s'agit, d'une part, d'un script gérant l'automatisation des déplacements des ennemis entre plusieurs points prédéfinis dans l'environnement de jeu, et d'autre part, d'un script assurant le suivi du personnage principal par les ennemis. L'implémentation de ces scripts d'intelligence artificielle dans le jeu s'est avérée particulièrement fastidieuse, car les codes initialement créés ne convenaient pas totalement à l'utilisation souhaitée.

Un important travail de débogage a donc été nécessaire afin de réaliser les connexions requises entre les différents composants du système, notamment pour permettre la prise de dégâts du personnage principal par les ennemis de manière cohérente. Un important travail de coordination et d'harmonisation des codes a dû être fourni pour faire fonctionner l'ensemble de manière interconnectée et fluide. Concernant le deuxième script C# relatif à l'intelligence artificielle, celui-ci n'avait pas encore été implémenté dans le jeu car les niveaux les plus difficiles n'ont pas encore été créés. Néanmoins, ce script a été testé à de nombreuses reprises dans des fichiers de test autonomes et fonctionne parfaitement, étant ainsi prêt à être intégré dès que les conditions le permettront.

3 Fonctionnalités et déroulé du jeu

Nous allons désormais synthétiser les différentes fonctionnalités du jeu ainsi que son déroulé.

Au lancement de l'application, le joueur arrive à l'écran d'accueil à partir duquel il pourra lancer une nouvelle partie. Dès lors, l'animation racontant le synopsis se lance pour immerger le joueur dans l'univers d'"Objectif Lune!". Après cette animation, le joueur sera amené dans une autre scène où il devra choisir entre activer le mode multijoueur ou pas. Le bouton "Hébergeur" permet de lancer une partie en tant que chef de la partie. Dans cette fonctionnalité, il pourra accueillir un autre joueur dans sa partie. L'autre choix est donc celui de rejoindre la partie d'un autre joueur en entrant le code d'accès de son partenaire.

Une fois le choix effectué, le joueur arrive dans le bunker. Celui-ci regroupe l'inventaire qui stocke les matériaux, un ordinateur où le joueur retrouvera les tutoriels du jeu et une carte permettant la sélection des différents niveaux. Le bunker abrite également un atelier pour améliorer la fusée à partir des matériaux, mais aussi une échelle permettant au joueur de retourner à l'écran d'accueil.

Le joueur pourra interagir avec tous ces éléments en étant proche de ces derniers tout en appuyant sur les touches dédiées. Sur un clavier AZERTY,

- La touche E est associée à l'échelle, à la fusée et à l'inventaire
- La touche T est quant à elle associée à la carte

Toujours sur un clavier AZERTY, le joueur peut se déplacer avec les touches D pour aller à droite, Q pour aller à gauche et espace pour sauter. Mais il peut également attaquer avec la touche F. Ainsi, le joueur pourra faire le premier niveau avec ou sans l'aide d'un autre joueur qui a toujours la possibilité de le rejoindre.

À la fin de chaque niveau, le joueur se verra accorder des récompenses qui l'aideront à l'amélioration de la fusée. Cependant, il est important de souligner que les niveaux ne se débloquent qu'à condition d'avoir terminé le précédent. Ce n'est qu'alors que le joueur pourra alors accéder au niveau suivant. Il a également le choix de refaire un niveau déjà réussi s'il le souhaite même s'il ne gagne pas de nouveau les récompenses. Les types de niveau sont variés : fuite, énigme, labyrinthe, exploration classique. De plus, les niveaux se déroulent de jour ou de nuit. Ces cycles jour/nuit peuvent être intéressants à connaître car ils définissent les types de monstres s'y cachant, certains préférant le jour et d'autres la nuit. Les connaître peut alors permettre au joueur de prédire les comportements des ennemis, chaque type d'ennemis ayant ses propres déplacements.

Comme dans tout bon jeu, le dernier niveau est l'ennemi final à combattre avant d'accomplir l'objectif du jeu : construire une fusée pour rejoindre la Lune. Cet ennemi final possède des capacités différentes que les simples ennemis rencontrés dans les niveaux. Il n'est donc pas à prendre à la légère.

4 Présentation des tâches qui ont été réalisé

4.1 Mélody

Pour cette dernière soutenance, il a fallu terminer les tâches qui ont été réalisées.

Tout d’abord, je me suis occupée de la finalisation du site internet. Le site internet a été donc codé en HTML, en CSS et en JavaScript. Nous pouvons y retrouver le synopsis du jeu, de nombreuses informations concernant le jeu, l’historique de projet, une présentation de l’entreprise et les problèmes accompagnés de leurs solutions. Mais nous y retrouvons aussi la possibilité de télécharger les différents rapports de projet et également la possibilité de télécharger le jeu ! Nous retrouvons également les sources utilisées pour réaliser le projet.

Le site étant “fait main”, nous avons essayé de donner au site une impression d’être immergé dans le jeu avant même de l’avoir téléchargé. Le site internet reflète donc notre jeu. Et pour ce faire, nous avons utilisé la même police utilisée dans le jeu ainsi que sur la jaquette. Les couleurs utilisées sont également présentes sur le site ainsi que certains visuels de jeu pour permettre d’attirer la curiosité des futurs joueurs. La mise en place de vidéo sur le site permet aux joueurs d’être informés sur le fonctionnement du jeu, mais aussi la manière dont le jeu va se dérouler en grande partie.

La mise en page du site est identique aux soutenances précédentes. Nous y retrouvons une barre de navigation située tout en haut du site. Lorsque nous sommes sur une page ou que nous passons la souris sur une catégorie, la case se blanchit.

Sur la page d’accueil, nous retrouvons le synopsis du jeu ainsi que certains graphismes du jeu tels que Sam, le personnage principal, les ennemis, mais aussi une courte vidéo du jeu.

Sur la page, “Qui sommes-nous”, nous retrouvons une description de l’entreprise avec une photo et une citation de chacun des membres. L’importance de connaître qui se cache derrière la création du jeu est cruciale. En effet, nous avons tous un parcours, une histoire et des connaissances différentes. Lorsque nous cliquons sur le nom d’un membre, celui-ci nous redirige vers son profil LinkedIn.

Ensuite, sur la page “Historique du projet”, nous retrouvons l’avancement du projet. J’ai donc réalisé un système à l’aide de JavaScript, qui, lorsque nous appuyons sur un mois, les tâches réalisées durant ce mois s’affichent. La difficulté dans la conception de ce système a été d’apprendre un nouveau langage de programmation en peu de temps en étant seule face à mon ordinateur. Cependant, JavaScript et C# ont de fortes similitudes, donc j’ai mieux

appréhendé.

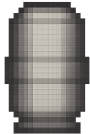
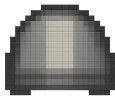
Un projet rencontre toujours des problèmes qu'ils soient réalisés seuls ou à plusieurs, les problèmes peuvent être divers. C'est pour cela que nous avons créé une page consacrée à nos problèmes accompagnée de leurs solutions que nous avons prises. L'importance de voir nos problèmes reflète notre acharnement et notre motivation concernant le projet.

Par ailleurs, nous avons une page dédiée aux différents rapports que nous avons dû réaliser ainsi que quelques diaporamas qui ont pu accompagner nos récits lors des soutenances. Une page est également dédiée pour le téléchargement du jeu et une autre pour les différentes sources utilisées pour réaliser le projet. De plus, dans chacune des pages, lorsque nous arrivons en fin de page, nos contacts s'affichent tels que l'adresse ainsi qu'une adresse mail pour nous contacter.

Pour que le site internet soit accessible à tous, j'ai décidé de mettre le site sur GitHub. GitHub a mis en place une capacité à réaliser une page internet qui permet aux utilisateurs d'avoir accès au site via un lien. Cependant, il n'est pas accessible lorsque nous recherchons sur un navigateur sans le lien. J'ai rencontré un premier problème mineur lorsque je souhaitais ouvrir mon site sur GitHub page. En effet, une erreur 404 s'affichait alors que j'avais mis tous mes fichiers sur GitHub. Cette erreur était générée puisqu'il ne trouvait pas le fichier "index.html". J'ai donc renommé le fichier de la page d'accueil passant de "site.html" à "index.html". Le site a pu être accessible ensuite sans générer d'autres problèmes.

Par la suite, je me suis également occupée de gérer la sauvegarde des données. Pour ce faire, j'ai utilisé PlayerPrefs, une fonctionnalité d'Unity qui permet de sauvegarder des données directement sur le disque de l'utilisateur. J'ai décidé d'utiliser PlayerPrefs, car nous avons peu de données à enregistrer ce qui ne nécessitait pas d'un système de sauvegarde de données plus développé. Cependant, l'utilisation de PlayerPrefs a un inconvénient. En effet, il est possible de modifier facilement les données dans les fichiers. Or, la modification des données n'a aucun intérêt pour les joueurs puisque la clé principale du jeu est de s'amuser et de découvrir son histoire. Cela n'apporterait rien d'avoir, par exemple, 10 000 pièces d'un matériel.

La sauvegarde des données a pour but de stocker les quantités des 8 matériaux sur le jeu, mais également de sauvegarder les réussites des niveaux ainsi que les données globales d'une partie en cours. Tout d'abord, la sauvegarde des 8 matériaux avec sa quantité. Pour rappel, il existe 8 matériaux différents :

Matériaux	Quantité nécessaire à la fusée	Image du matériau
Étage de propulsion	1	
Réservoirs de carburant	3	
Moteurs-fusées	4	
Système de guidage et de navigation	1	
Boîtier en aluminium	1	
Coiffe	1	
Structure de la fusée	1	
Bouclier thermique	8	

Lorsque nous commençons une nouvelle partie, chacun de ces matériaux sera à 0. PlayerPrefs fonctionne comme un dictionnaire sur C#. C'est-à-dire que chacun des matériaux possède un nom (key), ainsi qu'une donnée chiffrée (value). Chaque niveau possède un nombre de matériaux prédéfini qui peut être obtenu. Voici la répartition des matériaux dans chacun des niveaux :

- Niveau 1 : Structure de la fusée x1, Bouclier thermique x2
- Niveau 2 : Réservoirs de carburant x1, Moteurs-fusées x1
- Niveau 3 : Coiffe x1
- Niveau 4 : Bouclier thermique x2
- Niveau 5 : Réservoirs de carburant x1, Moteurs-fusées x1
- Niveau 6 : Boîtier en aluminium x1, Bouclier thermique x1
- Niveau 7 : Réservoirs de carburant x1, Moteurs-fusées x1
- Niveau 8 : Système de guidage et de navigation x1, Bouclier thermique x1
- Niveau 9 : Moteurs-fusées x1, Bouclier thermique x2
- Niveau 10 : Étage de propulsion x1

Lorsqu'un niveau se finit, les matériaux obtenus sont ajoutés dans l'inventaire faisant augmenter son nombre.

J'ai utilisé PlayerPrefs pour faire le système de la map. Pour rappel, les niveaux sont accessibles sur une carte où nous retrouvons chacun des niveaux associés à un bouton. En effet, lorsque le niveau précédent n'est pas fini, nous ne pouvons pas accéder au niveau suivant. Par exemple, nous avons fini le niveau 4 et nous voulons faire le niveau 6. Cela n'est pas possible sans avoir fait le niveau 5. Le niveau 6 et les niveaux supérieurs sont donc bloqués. Pour cela, chaque niveau possède un nom et lorsque le niveau est fini, nous mettons le nom du niveau avec comme valeur 1. Lorsque nous initions une nouvelle donnée, la valeur chiffrée est toujours à 0.

J'ai donc procédé de sorte à ce que si la donnée chiffrée est 0, le niveau n'est pas validé, mais que si la donnée chiffrée est 1, le niveau est validé. Ce système m'a permis également de faire en sorte que si le niveau a déjà été validé une fois, les récompenses ne peuvent plus être obtenues. En effet, le joueur pourra refaire autant de fois un niveau fini, mais n'obtiendra les récompenses que la première fois.

Par ailleurs, PlayerPrefs a été utilisé pour savoir si une partie a déjà été commencée ou non. J'ai d'abord initialisé une nouvelle donnée "HasData". Si nous pouvons y récupérer une valeur, le joueur pourra alors récupérer les données précédentes. Sinon, une nouvelle partie pourra se lancer. Pour ce faire, nous allons voir si le PlayerPrefs contient la donnée "HasData".

Par la même occasion, je me suis occupée du menu. L'interface du menu possède 3 boutons. Le premier pour lancer une nouvelle partie, le second pour continuer une partie déjà existante et le troisième pour quitter le jeu. Lorsque le joueur souhaite cliquer sur le bouton "Continuer" mais qu'il n'a jamais commencé le jeu, un texte s'affiche pendant 10 secondes pour le prévenir qu'il doit commencer une nouvelle partie.

Dans le cas où le joueur aurait déjà commencé une partie, mais qu'il clique sur le bouton "nouvelle partie", l'interface du joueur changera et lui préviendra qu'il a déjà une partie en cours. Il aura le choix de soit recommencer le jeu entièrement, soit de continuer la partie actuelle. S'il souhaite recommencer le jeu, toutes les données sauvegardées précédemment seront supprimées. PlayerPrefs dispose d'une fonctionnalité permettant de tout supprimer en une seule ligne avec PlayerPrefs.DeleteAll(), ce qui est très pratique.

Pour rappel, Chloé et moi avons inversé nos tâches. Je m'occupe donc du bunker, quant à Chloé, elle s'est occupée des niveaux. J'ai donc finalisé le bunker. Le bunker contient la map, l'inventaire, une échelle pour retourner au menu ainsi qu'un atelier pour construire la fusée. Pour l'inventaire, aux soutenances précédentes, je n'avais ajouté que la possibilité d'interagir avec l'inventaire avec la touche E et qui ouvrait un panel lorsque le joueur était proche. Sur ce panel, j'ai rajouté les images des matériaux ainsi que leurs quantités. Leurs quantités se mettent à jour grâce à un script qui cherche le nom du matériau et donc sa valeur. J'ai alors relié le texte qui indique la quantité avec le script qui recherchera sa valeur. Ensuite, je me suis occupée du système d'amélioration de la fusée. À gauche de l'échelle, nous retrouvons un atelier avec lequel nous pouvons interagir en appuyant la touche E lorsque le joueur est proche. La fusée possède 3 évolutions, chacune de ces évolutions nécessite un certain nombre de matériaux. Cette fusée sera construite à gauche de l'atelier.

Evolution	Matériaux nécessaires
Première évolution	— Bouclier thermique x3 — Réservoirs de carburant x2 — Moteurs-fusées x2 — Structure de la fusée x1
Seconde évolution	— Coiffe x1 — Bouclier thermique x2 — Système de guidage et de navigation x1 — Réservoirs de carburant x1 — Moteurs-fusées x1
Troisième évolution	— Bouclier thermique x3 — Moteurs-fusées x1 — Étage de propulsion x1 — Boîtier en aluminium x1

Lorsqu'on interagit avec l'atelier, un panel s'ouvrira, affichant les matériaux nécessaires

avec un bouton pour améliorer la fusée. Si le joueur n’a pas assez de matériaux, un message s’affiche pendant 5 secondes prévenant le joueur qu’il lui manque des matériaux avant de revenir sur la liste de matériaux nécessaire. Si l’amélioration a lieu, les quantités de matériaux seront soustraites de l’inventaire.

Pour conclure, un `GameObject` possédant le script des données de l’inventaire a pu m’aider à faire les différents systèmes et à stocker les informations. Ce qui m’a permis de faire les différentes tâches sur le projet. L’utilisation de `PlayerPrefs` et sa compréhension a été très rapide. En effet, les fonctionnalités sont assez transparentes. J’ai principalement utilisé `PlayerPrefs.GetInt(“nom_de_la_donnée”)` pour récupérer une valeur en fonction de sa clé. Et `PlayerPrefs.SetInt(“nom_de_la_donnée”,0)` pour mettre une nouvelle valeur sur une clé en question. Lors d’un changement de valeur, il ne fallait surtout pas oublier de sauvegarder les changements avec `PlayerPrefs.Save()` sans lesquels les données ne seraient pas à jour.

Hengrui s’étant déjà lancé dans la conception du réseau sans même s’en rendre compte, je l’ai soutenu lors de sa finalisation. En effet, il avait déjà bien avancé. Reprendre son travail aurait été beaucoup trop long pour comprendre sa démarche. De plus, j’avais de nombreuses autres tâches à réaliser. Je l’ai donc seulement aidé à trouver les derniers éléments pour sa finalisation. Hengrui en parlera dans sa partie.

De plus, Chloé m’a aidé sur la partie visuelle des différentes interfaces du jeu. Chloé a été un point stratégique dans notre jeu. Nous ne nous rendons pas compte à quel point son travail nous permet d’attirer de nombreux joueurs.

4.2 Chloé

Afin de finir ce projet, j’ai commencé par réaliser les niveaux manquants en commençant par ceux qui m’inspiraient le plus soit les derniers niveaux, le neuvième et le huitième. C’est en réalisant le neuvième où j’ai eu l’idée de reprendre le code des allers-retours des ennemis pour l’appliquer à des plateformes. C’est de cette manière que j’ai créé les plateformes mouvantes dans notre projet.

Afin que le personnage principal ne traverse pas les plateformes, il a fallu leur ajouter des colliders portant le tag “Ground”. Les colliders automatiques intégrés aux `tilemap` lorsque l’on dessine les plateformes ne suffisent pas à faire tenir Sam.

En effet, dès la première soutenance, nous avons remarqué qu’il traversait le sol et les plateformes à certains endroits. Cela était dû au fait que les colliders intégrés aux `tilemap` se

créent individuellement et non globalement.

Autrement dit, chaque bloc formant le sol ou une plateforme possédait son propre collider et non un collider pour une plateforme entière et le sol en entier. Ce fait créait des trous que notre personnage pouvait traverser. D'où la nécessité d'utiliser le composant "Box collider 2D" qu'on peut ajuster à la main pour créer des collider globaux. Je me suis alors dit que nous pouvions très bien dessiner des murs sans pour autant mettre de collider pour créer des murs que le joueur pourra traverser et à l'inverse mettre des collider là où il n'y a pas de murs pour créer des murs invisibles. C'est de cette manière qu'est né le niveau 8, un niveau où le joueur est confronté à plusieurs reprises à trois chemins dont un seul mène au suivant et donc à la sortie.

Par la suite, j'ai dessiné le niveau 4 et 7 pour revenir à la création de niveaux plus simple pour le joueur et donc moins complexe pour moi. Le niveau 6 a donc été le suivant à créer.

Personnellement, je pense que le niveau 6 est un des plus complexes à finir. C'est un niveau de fuite, c'est-à-dire que les ennemis infligent deux fois plus de dégâts au joueur, mais aussi un niveau se déroulant de jour, les ennemis sont donc la limace et la grenouille, dont le niveau 6 en est rempli. Je l'ai créé en me disant que ce niveau serait fait pour tester les capacités d'observation et de saut du joueur. Les mouvements des grenouilles peuvent donc paraître imprévisibles pour les joueurs se précipitants. Ces dernières peuvent traverser des plateformes pour venir attaquer le joueur et sachant que les plateformes sont relativement espacées, il faut trouver le bon moment pour sauter sans se faire avoir.

Après la création du niveau 6, le niveau 10 me paraissait trop simple : il ne possédait ni murs invisibles ni plateformes mouvantes comparé aux niveaux précédents puisque je l'avais créé pour la deuxième soutenance.

J'ai donc pris la décision de le refaire afin de le compliquer un peu. Il ne me restait que peu d'inspiration après tous les autres niveaux que j'avais créés. J'avais déjà réalisé tout ce que je pouvais faire avec les plateformes que j'avais sous la main. Il fallait du nouveau. L'inspiration m'est venue en repensant au monde 8 de "Super Mario Bros Wii" et en reprenant le jeu des enfants "le sol est de la lave". D'où l'ajout d'une nouvelle palette exclusivement pour un dessin : celui de la lave. Le niveau 10 est donc un niveau où le but est de rester un maximum sur les plateformes, loin de la lave.

Bien entendu, personne n'a mentionné que ces plateformes resteraient statiques. La difficulté finale du niveau 10 me parut enfin à la hauteur d'un dernier niveau avant le grand

final.

Le niveau 11 est le dernier que j'ai créé de moi-même. Il a fallu chercher d'autres types de plateformes et de décors car il avait été prévu que ce dernier niveau se ferait en intérieur, dans un laboratoire abandonné.

Contrairement aux autres niveaux où la caméra n'est amenée qu'à se déplacer sur l'axe des abscisses, elle peut également se déplacer sur l'axe des ordonnées dans le niveau 11. Il a été dessiné de manière à durer trois fois plus longtemps que les autres niveaux pour deux raisons principales :

1. Il s'agit du niveau final, c'est donc la pièce maîtresse du jeu, celle qui déterminera l'avis final du joueur sur notre jeu. Il fallait donc tout donner pour ce dernier niveau, redoubler d'ingéniosité pour le piéger et lui faire prendre du plaisir.
2. La bande son choisie pour ce dernier niveau n'est pas forcément plus longue que les autres, mais son écoute intégrale participe grandement à la mise en place de l'ambiance cruciale d'un dernier niveau. Il fallait donc un niveau plus long que les autres afin de pouvoir profiter pleinement de la bande son.

Pour retarder au maximum le joueur, je me suis donc principalement servie de l'astuce expliquée au-dessus selon laquelle la caméra ne se déplace pas verticalement dans les autres niveaux. De cette manière, j'ai fait en sorte de dessiner le haut du niveau de manière à ce que le joueur pense qu'il doit aller vers la droite pour atteindre la fin du niveau et qu'il mourra s'il tombe d'un trou. Tout a été fait pour manipuler le joueur alors que la chambre du boss final se trouve en réalité tout en bas à gauche du niveau.

Bien entendu, le niveau a également été dessiné de sorte à ce que le joueur puisse rejoindre cette chambre peu importe le moment où il aura compris qu'il faut descendre.

Les niveaux 3 et 5 ont quant à eux été créés en collaboration avec Nael qui m'a aidée dans la réalisation du script et des questions.

Pour rappel, les niveaux 3 et 5 sont des niveaux de type énigme où il faut répondre correctement à la majorité des questions pour pouvoir les valider. Un nouveau personnage y est introduit : Arachnée, la reine des araignées.

Dans l'histoire, c'est elle qui pose des questions à Sam pour savoir s'il mérite de passer ou s'il doit se faire manger par les enfants d'Arachnée. La personnalité d'Arachnée a été développée pour ressembler à une femme perverse même si non attractive physiquement parlant.

Le concept de ces niveaux énigmes est de faire faire une pause intellectuelle au joueur tout en rappelant qu'il s'agit d'un jeu d'aventure. Ils permettent donc également de remettre le joueur dans la narration de l'histoire avant qu'il ne reprenne les niveaux classiques.

En ce qui concerne les graphismes, Coline nous a réalisé les dessins nécessaires à l'animation du début ainsi que ceux pour l'animation de l'attaque de Sam, ceux du boss final et pour finir l'illustration d'Arachnée et celle de notre jaquette.

J'ai ainsi pu reconstituer l'animation du début. Celle-ci m'a pris plus de temps que prévu : il a fallu activer et désactiver les plans au bon moment et s'occuper des mouvements de caméra et de certains objets pour ajouter du dynamisme dans l'animation. Le plus compliqué a été les mouvements fluides des dessins et de la caméra, mais aussi d'empêcher le joueur de passer au dessin suivant avant d'avoir complètement fini les animations.

Ces restrictions se sont traduites par des conditions dans un LateUpdate dans le script : le joueur n'a le droit de passer à la suite seulement si le dessin devant bouger à atteint sa destination.

Autrement dit, si la différence entre les vecteurs de la destination et du dessin est inférieure à 0,1 en moyenne. J'ai fait varier cette différence pour choisir le moment exact où le joueur peut passer au dessin suivant.

Toujours par rapport à l'animation du début, après avoir fait tous les réglages des mouvements et des superpositions des dessins, j'ai demandé à notre responsable du son de faire une sélection de bruitage afin de dynamiser les scènes les plus marquantes, comme l'explosion nucléaire. Nous sommes ensuite passés à leurs implémentations pour avoir le résultat le plus précis et cohérent avec notre animation.

J'ai, par la suite, retravaillé la jaquette que Nael avait commencée en reprenant son résumé et sa mise en page, comportant l'illustration de Coline. J'y ai ajouté la police d'écriture que nous avions préalablement sélectionnée, quelques images du jeu, le décor de nos niveaux de jours pour apporter de la cohérence avec la première de couverture et une touche d'esthétisme, la mention "uniquement disponible sur Windows" ainsi qu'un qr code menant

à notre manuel d'installation et d'utilisation.

L'animation de l'attaque du personnage principal a été implémentée en jeu par Hengrui, notre responsable du personnage principal. L'animation du boss final a été utilisée pour la fin du niveau 11 tandis que l'illustration d'Arachnée dans les niveaux 3 et 5.

4.3 Hengrui

Pour cette dernière soutenance, je me suis principalement occupé du multijoueur et du réseau et la dernière fonctionnalité manquante de notre personnage.

Concernant le personnage, la fonctionnalité que j'ai ajouté est l'attaque du personnage. Son implémentation pour le mode solo fut assez simple. Pour le code de l'attaque, j'ai d'abord d'ajouter un layer qui sera donné à tous les ennemies, un gameobject attaquepoint qui donnera la zone de l'attaque, mais aussi une autre variable qui donnera la taille de la zone qui pourra être touché, et un code dans le script de l'ennemi qui lui fait perdre ses points de vies. La logique derrière ce code est assez simple, lorsque l'attaque et son animation se lance, on va se servir de l'attaquepoint pour détecter si un gameobject possédant le layer ennemi est touché, si c'est le cas, alors on sait que c'est un ennemi, donc on appelle le script qui fait perdre la vie à l'ennemi, et lorsque leur point de vie tombe à 0, il est détruit.

Pour l'implémentation de ce code en multijoueur, le seul problème que j'ai rencontré provenait de l'animation lorsque le jeu se lançait sur deux ordinateur différent, c'est à dire que sur l'ordinateur de l'hébergeur, les animations de l'hébergeur ou de l'hébergé sont visibles, cependant sur l'ordinateur de l'hébergé, les animations de l'hébergeur ne sont pas visibles. La raison de ce problème est que Unity ne synchronise pas automatiquement les animations entre les clients.

De ce fait, j'ai ajouté des NetworkVariable qui nous permettent de synchroniser manuellement les changements d'états. J'ai ensuite mis tous ces codes dans la méthode OnNetworkSpawn qui est appelée automatiquement par Netcode lorsqu'un objet réseau est instancié localement. Grâce à cela, le problème fut résolu assez facilement.

Dans le paragraphe précédent, j'ai parlé de pouvoir jouer au jeu sur des ordinateurs différents, c'est aussi une fonctionnalité que j'ai réalisée pour cette dernière soutenance. J'ai réalisé ceci avec Unity relay. La logique d'Unity relay est d'activer le host comme lorsqu'on utilise le networkmanager, après son activation, on obtient un code qui pourra être utilisé par les autres personnes pour rejoindre la partie. Et donc pour la réalisation d'Unity relay,

je suis tout d'abord aller sur Unity Cloud pour activer le relay sur notre projet Unity.

Après son activation, j'ai créé une scène spéciale qui possède deux boutons, celui pour l'activation de l'hébergeur, et celui pour rejoindre le jeu. Dans cette scène spécial, deux autre script peuvent être retrouvé, tout d'abord le code qui permet de changer la scène lorsque l'hébergeur est lancé, mais aussi le code qui permet de garder le code qui permet à l'hébergé de rejoindre, et c'est grâce à ce code que j'ai réussi à afficher le code qui permet de rejoindre la partie dans les autres scènes.

Après les réalisations de ces différentes tâches, je me suis tourné vers la modification des codes, pour qu'ils puissent tous fonctionner en mode multijoueur. Comme nous sommes dans un jeu de plateforme, l'apparition automatique du personnage lorsqu'on passe d'un niveau est bien entendu obligatoire, donc j'ai commencé par la réalisation de ce code.

Pour la réalisation de ce code, la logique utilisée est de faire apparaître, dans un premier temps tous les personnages présents dans le jeu, puis les téléporter à une position précise que nous avons définie à l'aide d'un gameobject que nous avons placé.

Cependant, le gros problème que j'avais rencontré est que la téléportation ne se réalisait pas : à chaque fois, les personnages apparaissaient à la position (0,0,0), la position par défaut.

C'est pourquoi j'ai voulu tester le code de la téléportation. Le code ne fonctionnait pas, j'ai donc passé énormément de temps dessus pour essayer de trouver le problème.

Finalement, le problème venait en réalité d'une activation trop rapide des fonctions qui permettaient de téléporter les personnages à la bonne position, car elles activaient avant même que les personnages apparaissent dans la scène. La téléportation téléporte donc le vide.

Pour résoudre ce soucis, j'ai donc ajouté des codes comme `WaitForSeconds` qui permet d'ajouter un délai avant l'activation du code de la téléportation, ce qui a résolu le problème. Grâce à ce délai, tous les joueurs ont le temps d'apparaître, et la fonction téléporte donc bien tous les joueurs à la bonne position.

Une autre fonctionnalité est que la vie du personnage doit être remise à 100 lorsqu'il change de niveau. Cependant, au début, ce n'était pas le cas dans notre jeu, même si on changeait de niveau, les points de vie n'étaient pas réinitialisés. Cette fonctionnalité a pu

être réalisée en partie grâce à cette apparition automatisée.

Pour réaliser cette fonctionnalité, j'ai tout d'abord créé un code qui permet de remettre la vie à 100 dans le code du personnage, l'ai appelée dans le spawnManager (script qui réalise l'apparition automatique). C'est-à-dire que lorsqu'on fait apparaître les différents personnages dans les niveaux, on réinitialise leur barre de vie, puis on les téléporte à la bonne position.

De ce fait, la fonctionnalité de réinitialiser la barre de vie quand on passe à un autre niveau a pu être réalisée.

Au niveau des codes déjà présents comme ceux des ennemis, j'ai dû les modifier pour qu'ils puissent fonctionner en multijoueur. J'ai également dû modifier l'affichage du panel lorsque le joueur perd.

Avant le changement, ce panel s'affichait lorsque le personnage est mort, et un bouton recommencer et un bouton retour au bunker apparaît. Cependant le bouton recommencer ne fonctionnait pas, donc durant la réalisation du code, j'ai aussi implémenté la même méthode que dans le spawnmanager, un script qui permet de faire apparaître le personnage est de le téléporter au point de départ.

Après la réalisation de cela, je me suis tourné vers l'affichage de ce panel game over, mais seulement lorsque tous les personnages sont morts. Le script avant changement lance le panel du game over lorsque le personnage meurt, donc si le host il mourrait, le panel s'affichait dans les deux écrans, et le niveau ne pouvait plus être continué.

Donc dans le script du game over, j'ai ajouté un code qui permettait donc de vérifier tout d'abord si tous les personnages présents sont morts. Si c'est le cas, alors le panel du game over s'affiche dans les deux écrans.

Cependant, le problème persistait, le code n'avait pas de problème, cependant lorsque l'un des deux joueurs meurt, le panel du game over s'affiche quand même directement. J'ai donc fait des recherches, et trouvé la source du problème. Le problème est que le panel et le gameovermanager était attaché au prefab du personnage, alors qu'il fallait le mettre dans les différentes scènes. Et ceci étant la source du problème, après avoir déplacé le gameovermanager et le panel dans les différentes scènes, le problème fut résolu.

Pour le changement de scène, remplacer `Scenemanager.loadscene` en `Networkmanager.scenemanager.loadscene` fut suffisant.

Cependant, au niveau de ce changement de scène, il restait un problème majeur, le fait que le client ne puisse pas changer la scène même en appuyant sur les bonnes touches ou boutons. C'est un gros problème, car chaque niveau peut se finir de deux manières différentes, soit on termine le niveau et on passe au prochain niveau, soit tous les personnages meurent et on peut recommencer le niveau.

Cependant, dans le cas, où l'hébergeur est le seul joueur qui est mort, si le client n'est pas en capacité de changer de scène alors, il est obligé de se suicider, et ce n'est pas le but de notre jeu, le client doit aussi avoir le pouvoir de terminer le niveau même si l'hébergeur est mort. De ce fait, l'ajout de cette fonctionnalité est primordial. Et en réalité, cette fonctionnalité n'est pas difficile à mettre en place.

Chez Unity, on utilise souvent `ServerRPC` qui permet au client d'envoyer une commande au serveur, cependant, en général on écrit `[ServerRpc]` qui est équivalent de `[ServerRpc(RequireOwnership = true)]`, le problème de cette écriture est qu'il autorise seulement au client possédant le `networkobject` (hébergeur) d'envoyer des commandes au serveur.

En comprenant ceci, la réalisation de cette fonctionnalité était vraiment rapide et simple, il m'a simplement suffit d'écrire `[ServerRpc(RequireOwnership = false)]`, et le problème fut résolu. Cette ligne de code permet de réaliser cette fonctionnalité car en mettant le `RequireOwnership` en `false`, l'envoi des commandes ne se limitent plus au hébergeur, mais tous les clients peuvent envoyer les commandes et seront reçus par le serveur, ce qui est totalement ce qu'on cherche à réaliser.

Cette ligne de code `"[ServerRpc(RequireOwnership = false)]"` a été utilisée de nombreuses fois dans les scripts pour pouvoir réaliser cette fonctionnalité dans de nombreuses situations.

Un autre problème que j'avais rencontré est le fait que les panels se lançaient seulement chez l'ordinateur du host. La raison de ce problème est que seule la machine qui exécute le code va voir le panel, donc par conséquent le host.

Cependant, ce problème est assez simple à régler, l'ajout de la ligne `[ClientRPC]` est suffisant. En effet, `ClientRPC` permet au serveur d'envoyer un message réseau à tous les clients connectés, et en envoyant ce message tous les clients vont exécuter le code qu'exécute le host,

ce qui résout parfaitement notre problème. Cette méthode a été utilisée dans de nombreux script comme l’affichage du panel de la victoire par exemple.

4.4 Nael

En ce qui concerne l’implémentation de l’intelligence artificielle (IA), celle-ci se concentre principalement sur l’automatisation des mouvements et des comportements des ennemis que le joueur rencontre tout au long de son aventure. Cette approche novatrice permet d’améliorer considérablement l’expérience de jeu en rendant les interactions avec les ennemis non seulement plus dynamiques, mais également beaucoup plus réalistes et engageantes.

En effet, l’IA joue un rôle crucial dans la manière dont le joueur perçoit et interagit avec son environnement, transformant ainsi chaque rencontre en un moment mémorable et palpitant. Grâce à cette technologie, nous avons pu créer un monde de jeu où chaque ennemi semble avoir sa propre personnalité et ses propres motivations, ce qui enrichit l’immersion du joueur.

Le premier script que nous avons développé gère l’automatisation des déplacements des ennemis entre divers points prédéfinis dans l’environnement de jeu. Ce système de patrouille permet aux ennemis de se déplacer de manière fluide et prévisible, créant ainsi un défi intéressant pour le joueur.

En observant leurs mouvements, le joueur peut anticiper leurs actions, ce qui ajoute une couche stratégique au gameplay. Cette mécanique de patrouille, bien que simple en apparence, est essentielle pour établir un rythme de jeu captivant et pour encourager le joueur à élaborer des stratégies afin de surmonter les obstacles.

Cependant, pour enrichir davantage l’expérience, nous avons décidé de modifier notre approche pour le second script, cherchant à offrir quelque chose de plus captivant et imprévisible.

Comme vous pouvez le remarquer, un changement significatif a été opéré concernant le deuxième script de déplacement. Au lieu de maintenir une simple patrouille, nous avons choisi d’implémenter un script qui crée l’illusion de sauts de grenouille. Cette décision a été motivée par notre désir d’améliorer l’immersion dans le jeu, en donnant l’illusion que l’ennemi effectue des sauts dynamiques et inattendus, semblables à ceux d’une grenouille.

Cette approche non-conventionnelle vise à surprendre le joueur et à rendre les interactions encore plus palpitantes. En intégrant ces mouvements imprévisibles, nous avons réussi à créer un environnement de jeu où le joueur doit constamment s'adapter et réagir, augmentant ainsi le niveau de tension et d'excitation. Bien que cette intégration ait présenté des défis techniques, le script fonctionne désormais de manière fluide et efficace, ajoutant une dimension ludique à chaque rencontre.

Je n'ai pas rencontré de réels problèmes majeurs lors du développement, à l'exception d'un défi particulier lié à l'ancien script de suivi total du joueur par les ennemis. Malgré un travail de recherche intensive et plusieurs tentatives d'adaptation, ce script n'a pas pu être intégré de manière définitive dans le jeu.

À l'origine, ce script avait été conçu pour permettre aux ennemis de suivre le joueur à travers toute la carte, dans le but d'ajouter une couche de difficulté supplémentaire aux niveaux, rendant ainsi l'expérience de jeu plus engageante et stimulante.

Cependant, un problème significatif est survenu lors des essais d'implémentation du script dans le jeu. En effet, l'ennemi qui était censé suivre le joueur se retrouvait systématiquement à retourner au même point de départ, ce qui était non seulement frustrant, mais aussi contre-productif pour le gameplay.

Après une analyse approfondie, il est apparu que la cause de ce comportement inattendu résidait dans le fait que le joueur devait cliquer sur un bouton spécifique pour faire apparaître notre personnage. Par conséquent, la position du joueur, telle qu'elle était perçue par le code de l'ennemi, ne changeait pas réellement.

En d'autres termes, le script ne pouvait pas détecter les mouvements du joueur de manière efficace, ce qui entraînait un comportement erratique où l'ennemi rejoignait inexorablement le point de départ du joueur.

Face à cette situation, nous avons pris la décision, comme expliqué précédemment, de changer totalement de script. Cette réévaluation nous a permis d'explorer d'autres solutions plus efficaces et adaptées aux besoins du jeu.

En fin de compte, cette expérience nous a enseigné l'importance de tester minutieusement chaque élément du code et de s'assurer que les interactions entre les différents composants fonctionnent harmonieusement pour garantir une expérience de jeu fluide et agréable.

Ainsi, bien que ce défi ait été frustrant sur le moment, il a également ouvert la porte à de nouvelles perspectives et à des approches plus innovantes dans le développement de notre jeu. Nous avons pu tirer des leçons précieuses de cette situation, ce qui nous a permis d'améliorer non seulement le suivi des ennemis, mais également d'autres aspects du gameplay, enrichissant ainsi l'expérience globale du joueur.

Pour réaliser cette illusion, nous avons utilisé un mouvement parabolique, ce qui a nécessité l'application d'une fonction sinusoïdale. En partant de la base du script initial qui gère les aller-retour, nous avons également intégré la fonction `Vector3.Lerp` pour garantir un mouvement fluide et naturel de l'ennemi, rendant ainsi l'expérience de jeu encore plus captivante.

Ce niveau de détail dans le mouvement contribue à créer une atmosphère immersive où le joueur peut véritablement ressentir la tension et l'excitation de chaque confrontation. En effet, chaque saut de l'ennemi est calculé avec précision, ce qui donne une impression de réalisme et d'authenticité, renforçant ainsi l'engagement du joueur.

En parallèle, je me suis également occupé de la création d'une fonction permettant à l'ennemi de se retourner sur l'axe "x". Cette fonctionnalité contribue à un réalisme accru concernant le mouvement des ennemis.

En effet, après l'implémentation de ce programme sur les scripts des ennemis, ceux-ci sont désormais capables de regarder dans la direction où ils se déplacent. Cela permet une meilleure cohérence et fluidité dans l'interaction entre le joueur et les ennemis, rendant les rencontres plus engageantes et dynamiques.

Ce réalisme accru améliore non seulement l'esthétique du jeu, mais renforce également l'engagement du joueur, qui doit constamment rester sur ses gardes. Cette attention aux détails contribue à créer une expérience de jeu immersive où chaque mouvement compte.

J'ai également participé à la réalisation de plusieurs niveaux du jeu, en particulier les niveaux 3 et 5, qui sont des niveaux basés sur des questions à choix multiples.

Pour cela, j'ai dû créer un script permettant le bon fonctionnement de ces niveaux. La tâche n'a pas été de tout repos, car nous avons dû passer par plusieurs phases de réflexion

et de conception avant de pouvoir créer un script efficace. Ce dernier repose sur un système de tableau prédéfini et pré-rempli afin que le code créé en C# puisse dispatcher d'une part les questions au bonne zones de texte et d'une autre part les 3 réponses possibles, c'est donc le joueur qui choisit sa vitesse d'exécution permettant ainsi au joueur de gérer la vitesse des questions posées.

Afin que le script comprenne que les boutons ont été cliquer et qu'il faut lancer la fonction la ligne suivante a été utilisé : (`answerButtons.onClick.AddListener(() => AnswerButtonClicked(index));`) cette ligne permet au bouton d'activer l'appel d'une fonction permettant la vérification de la réponse proposé.

De plus, les réponses sont disponibles pour aider le joueur en cas d'erreur, offrant ainsi une expérience d'apprentissage interactive qui encourage la réflexion et la stratégie.

Les niveaux 3 et 5 sont considérés comme réussis si le joueur parvient à terminer toutes les questions en commettant moins de trois fautes.

Dans le cas contraire, le joueur est autorisé à rejouer jusqu'à atteindre le score requis, ou il peut choisir de retourner au bunker pour se préparer à de nouveaux défis. Pour faciliter cette navigation, un système de chargement de scène a été mis en place, garantissant une transition fluide entre les différents niveaux.

Ce système améliore non seulement l'expérience utilisateur, mais permet également de maintenir un rythme de jeu soutenu et agréable. La manière de vérifications des réponses posées se fait aussi au préalable, c'est en effet un tableau de réponses pré-rempli qui est utilisé comme moyen de vérification de réponse avec l'utilisation de comparaison d'index.

Par la suite, chaque niveau est conçu pour être à la fois stimulant et accessible, afin que le joueur puisse progresser à son propre rythme tout en étant toujours poussé à s'améliorer.

Enfin, j'ai participé à l'implémentation des bandes sonores et des effets auditifs durant la cinématique d'introduction. J'ai eu l'opportunité d'ajouter des effets sonores immersifs, tels que des explosions, des bruits de métro, et d'autres éléments sonores qui contribuent à l'atmosphère générale du jeu.

Ces éléments sonores sont cruciaux pour plonger le joueur dans l'univers du jeu et ren-

forcer l'impact émotionnel des scènes.

Par ailleurs, j'ai également contribué à la réalisation de la jaquette de notre jeu vidéo, veillant à ce qu'elle reflète l'essence et l'intensité de l'expérience de jeu que nous avons créée. Ce travail graphique est essentiel, car il constitue la première impression que le joueur aura de notre projet, et nous avons voulu qu'il soit à la hauteur de nos ambitions créatives.

En somme, chaque aspect de notre jeu a été soigneusement pensé pour offrir une expérience mémorable et immersive, et je suis fier de ce que nous avons accompli ensemble.

Pour le son :

Scène	Description
Bunker	Ce morceau, intitulé "Bunker", évoque les musiques d'attente qui instaurent une ambiance apaisante pour le joueur. En effet, le bunker est un lieu où le personnage principal doit se sentir à l'aise et en sécurité. Ce choix musical vise à créer une atmosphère tranquille, propice à la réflexion et à la détente, permettant ainsi au joueur de s'immerger pleinement dans l'univers du jeu.
Le menu du jeu	Le morceau qui accompagne le menu du jeu se distingue par ses sonorités joyeuses et attractives. On pourrait qualifier ce son d'accueillant, car il invite le joueur à s'engager dans l'expérience ludique. Cette musique légère et optimiste sert à établir un premier contact positif, créant une ambiance conviviale qui donne envie de plonger dans l'aventure.
Niveaux 1, 2, 6, 8, 10 - Fight with Devotion	Les morceaux associés aux niveaux 1, 2, 6, 8 et 10 sont entraînants et contribuent à créer une atmosphère immersive et chaleureuse. Ils sont idéaux pour favoriser la détente et la concentration, permettant au joueur de se plonger dans le gameplay sans distraction. Ces sons dégagent une sensation de nostalgie et de mélancolie, tout en restant apaisants, ce qui enrichit l'expérience émotionnelle du joueur.
Niveaux 3 et 5 - Roam	La piste audio Roam dégage une atmosphère intrigante et immersive, évoquant des émotions de nostalgie, d'optimisme et de réflexion. Ce choix de bande sonore pour les niveaux 3 et 5 a été fait avec soin, car ces niveaux sont conçus comme des quiz où la réflexion est primordiale pour triompher. La musique accompagne le joueur dans son parcours intellectuel, créant un environnement propice à la concentration et à l'analyse.

Niveau 4 - Fight 150 BPM	Le morceau "Fight" à 150 BPM crée une atmosphère entraînante qui pousse à l'action et à la lutte. Il évoque des émotions de suspense et de mystère, captivant le joueur dans un tourbillon d'adrénaline. On peut également relever l'utilisation de solos instrumentaux, qui ajoutent une dimension de mystère à la musique. Cette bande sonore est utilisée dans les niveaux où l'intensité et la tension sont à leur comble, notamment dans des séquences de fuite nocturne, renforçant ainsi l'immersion dans l'action.
Son 6 : Niveaux 7 et 9 - Trouble	Le morceau "Trouble" est particulièrement entraînant, avec une grande partie rythmique jouée à la batterie, qui dynamise l'expérience de jeu. Cette bande sonore évoque une atmosphère d'inquiétude et de mystère, tout en suscitant par moments des émotions de nostalgie et d'optimisme. Dans le niveau 7, qui se déroule lors d'une exploration nocturne, cette musique accompagne le joueur dans sa quête, créant une tension palpable tout en favorisant une connexion émotionnelle avec le monde du jeu.
Niveau 11 - challenger	Le son évoque un niveau épique à venir nous pouvons y retrouver plusieurs parties ou connotations à certains sons et bande sonores du jeu, mais cette musique évoque aussi une émotion de suspense haletant. Il représente l'apogée de l'expérience musicale du jeu, combinant des éléments de suspense et d'excitation. Ce son unique est destiné à marquer la conclusion du parcours du joueur, lui offrant une dernière poussée d'adrénaline et d'engagement avant de terminer l'aventure.

5 Problèmes rencontrés lors de la réalisation du projet

La réalisation d'un projet est toujours accompagnée de défis et de problèmes, qu'il soit entrepris seul ou en équipe.

En effet, chaque étape de cette réalisation est une occasion d'apprentissage et de développement, tant sur le plan technique que personnel. C'est lors de ce processus que se forment de nouvelles compétences et que se créent des rencontres enrichissantes.

En faisant face à ces difficultés, nous découvrons notre capacité à nous adapter et à surmonter les obstacles, ce qui nous rend encore plus résilients.

Dans notre cas particulier, nous avons débuté le projet avec un bagage de connaissances relativement limité, et pour certains, il n'y avait même aucune expérience préalable.

Cependant, ce manque de connaissances initiales n'a pas constitué un frein à notre détermination.

Au contraire, nous avons fait le choix conscient de nous engager pleinement dans ce projet, malgré les divers obstacles que nous avons rencontrés en cours de route. Les problèmes auxquels nous avons été confrontés varient en complexité.

Certains étaient des défis relativement simples, des situations communes que beaucoup peuvent rencontrer dans le cadre d'un projet.

D'autres, en revanche, étaient moins évidents et nécessitent une réflexion plus approfondie et une approche créative pour être résolus.

Ainsi, nous allons vous présenter en détail les différents problèmes que nous avons rencontrés tout au long de notre parcours, ainsi que les solutions que nous avons mises en place pour les surmonter.

	Problème	Solution
Synchronisation des différentes branches	Initialement, nous devions utiliser Gitlab pour le projet. Chacun des membres du groupe possédait une branche dans laquelle nous faisions nos tests pour ensuite fusionner les branches et créer notre jeu. Cependant, nous avons eu des difficultés pour fusionner cela ne fonctionnait pas comme nous le souhaitions. Le fond de chacune des scènes se retrouvaient en fuschia alors qu'il devait s'agir d'un dessin réalisé par notre graphiste.	Passer les fichiers sur un drive pour les rassembler sur un seul ordinateur et relier tous les bons paramètres à la main. Nous sommes par la suite passés par Github Desktop qui est plus facile en termes d'utilisation et qui ne génère pas de nouveau problème.
Github Desktop	L'utilisation de Github Desktop nous a grandement aidées pour la réalisation de ce projet. Mais nous avons rencontré un souci majeur lors de son utilisation. Dans les fichiers modifiés, nous retrouvions régulièrement des fichiers qui se modifiaient seuls sans que nous ne modifiions quoi que ce soit dessus. Ce fichier était un asset qui nous permettait de changer la police d'écriture. Cette modification de fichier nous a causé à tort, lorsque nous soumettions le projet dans le répertoire, celle-ci causait des problèmes sur le projet des personnes qui tentait de le récupérer.	Nous avons utilisé la commande suivante : <code>git checkout - "nom_du_fichier"</code> . Cette commande permet de restaurer des fichiers de l'arbre de travail. Donc les fichiers qui se modifiaient seuls, se remettaient à l'état initial c'est-à-dire sans modification.

Affichage du panel "Game Over"	Si le joueur ne possède plus de vie ou qu'il tombe dans un trou, un panel "Game Over" doit s'afficher. Or ce n'était pas le cas.	Le panel "Game Over" était relié au joueur qui, lui, possédait un <code>gameObject.Destroy</code> , ce qui détruisait tout ce qui était relié à lui. En séparant le joueur du panel "Game Over", le panel a pu s'afficher correctement.
Synchronisation des mouvements des monstres	Lors d'une session multijoueur où deux joueurs se retrouvent sur un même niveau, les mouvements des monstres doivent être synchronisés, c'est-à-dire que les mêmes mouvements doivent être faits à la seconde. Or ce n'était pas le cas, les mouvements étaient décalés.	Pour régler ce problème, il a fallu ajouter les composants <code>Network Transform</code> et <code>Network Object</code> .
Synchronisation des animations entre deux ordinateurs	Lorsque le jeu se lance sur 2 ordinateurs différents, l'animation du host n'est pas visible sur l'ordinateur du celui qui n'est pas le host, même si cette animation se lance bien sur l'ordinateur du host.	L'ajout des <code>NetworkVariable</code> qui nous permet de synchroniser manuellement les changements d'états a permis de résoudre ce problème. Tous ces codes ont été mis dans la méthode <code>OnNetworkSpawn</code> qui est appelée automatiquement par <code>Netcode</code> lorsqu'un objet réseau est instancié localement.
Superposition des barres de vie en multijoueur	Lors d'une session multijoueur où deux joueurs se retrouvent dans un même niveau, il y a une superposition des barres de vie ce qui fait que lorsque le joueur perd de la vie, cela ne s'affiche pas.	L'ajout d'une condition qui désactive la barre de vie de celui qui n'est pas le joueur dans le code de l'affichage de la barre de vie fut suffisant pour résoudre ce soucis.

Suivi de l'ennemi	Lors d'un niveau, le suivi de l'ennemi ne fonctionnait pas étant donnée que pour le jeu, le tag du joueur ne bougeait pas donc par conséquent l'ennemi rejoignait toujours le même point.	Pour régler ce problème, nous avons décidé de supprimer et de remplacer le script du saut de grenouille par un script qui a été modifié.
Mise à jour des récompenses	Lorsque nous souhaitions ajouter les récompenses de fin de niveau, l'inventaire ne se mettait pas à jour et affichait continuellement 0 pour chacun des matériaux.	Pour régler le soucis, il a fallu ajouter le GameObject de l'inventaire dans chacun des niveaux. Il a permis de récupérer la quantité actuelle de matériaux pour le mettre à jour en ajoutant les quantités obtenues.
Mise en ligne du site internet sur GitHub Page	Lors du dépôt de tous les fichiers sur un répertoire sur GitHub et que nous lançons le site, une erreur 404 s'affichait alors qu'en local, il n'y avait aucune erreur.	Il a fallu avoir un fichier "index.html", c'est ce que GitHub recherchait dans les fichiers. Or, il n'était pas présent. Pour ce faire, j'ai modifié le nom de mon fichier principal qui est passé de "site.html" à "index.html".
Affichage de la fusée	Dans notre jeu, nous avons la possibilité de faire apparaître visuellement chaque étape de l'évolution de la fusée. En effet, le système d'amélioration de la fusée marche. Il affiche bien la quantité demandée et la quantité est soustraite de l'inventaire. Or, dès le lancement d'une partie, la fusée est déjà construite entièrement. Ce qui ne devrait pas être le cas.	Cela a été dû à une mauvaise gestion de l'index sur l'affichage des fusées. Les images sont désactivées au début. J'ai donc dû mettre à jour le script en faisant en sorte que l'affichage de la première fusée ne soit affiché qu'une fois l'amélioration soit faite. En alternant avec l'index -1 et 0, pour éviter de tout afficher d'un coup.

6 Ressenti du projet

Voici le ressenti de chacun des membres du groupe. Vous trouverez également un ressenti global du groupe.

Mélody :

“J’ai toujours eu des mauvaises expériences en travaux de groupe et de projet. L’année dernière, j’ai été à l’université où il n’y avait pas de projet en groupe prévu.

Cela faisait donc un an que je n’avais pas travaillé en coopérant avec des personnes. J’appréhendais donc ce projet qui durait toute l’année et qui se basait sur un sujet que je ne maîtrisais pas.

De plus, nous n’avions aucun cours de prévu concernant Unity et nous devions donc faire des recherches par nous-même.

Malgré tout, je suis tombée sur des personnes agréables avec qui je me suis vite entendue et c’est ainsi que nous avons formé Dev&Co. J’ai toujours été une personne organisée, qui stresse énormément, et qui veut tout faire dans les temps.

Cependant, ce n’était pas le cas pour tous et j’ai donc dû m’adapter, ce qui n’était pas évident. Le projet n’a pas été évident. J’ai dû m’investir sur les cours, travailler à un petit job étudiant à côté, sur le projet bien évidemment et m’investir également dans une association dans laquelle je suis. L’organisation était donc primordiale pour ne pas être débordée. Il m’arrivait parfois de penser à tout abandonner. Mais je me suis rappelée que j’ai commencé EPITA dans le but de finir les 5 prochaines années qui m’attendent. C’est pour cela, que malgré les obstacles, les moments de doutes, le stress, rien ne m’arrêtera de continuer.

De plus, étant cheffe de projet, je n’avais aucune possibilité d’abandonner le projet. Je me suis proposée à être cheffe de projet dans le seul but d’assurer le bon déroulement. J’ai fait de mon mieux pour planifier des dates limite pour chacune des tâches pour assurer un bon suivi pour les soutenances.

Ce n’était pas évident puisque chacun travaillait à son rythme. J’ai donc dû trouver un équilibre qui convient à tout le monde.

Être cheffe de projet est un rôle qui me plaît énormément bien qu’un poids s’ajoute en

plus de mon rôle de membre du groupe.

Concernant le projet, je trouve que je me suis mal organisée, ce qui fait que je n'ai pas pu faire ce que nous voulions faire initialement et surtout de mieux implémenter ce que notre groupe souhaitait.

Malgré tout, j'ai pu acquérir de nouvelles capacités en communication et en projet de groupe. Mais ce projet a pu me confirmer que je ne souhaite plus travailler sur la création d'un jeu vidéo.

Le projet en lui-même ne m'a pas déplu, car je suis quand même fière de ce que nous avons pu produire. J'ai préféré la partie code, c'est-à-dire toutes les fonctionnalités qui se cachent derrière le jeu et non la partie visuelle du jeu. Si on me proposait de refaire le jeu, de me confronter à nouveau à ce projet, j'accepterais malgré tout puisque le projet renforce des liens et met aux défis nos capacités. Le challenge est une sensation qui donne envie d'être meilleur."

Chloé :

"Je diviserai mon avis du projet en deux parties : mon avis sur le groupe qu'est Dev&Co puis mon avis sur la réalisation du projet.

Dans un premier temps, mon avis sur le groupe. Les groupes de projet se font extrêmement tôt volontairement pour travailler avec des personnes que nous ne connaissons pas.

Malgré ce point, j'avais un assez bon pressentiment. Pressentiment qui s'est avéré être juste. En écoutant certaines personnes de notre classe, on pouvait comprendre que la plupart des groupes ne s'entendaient pas bien, entre des problèmes de communications, des abandons avant même la fin du premier semestre, des disputes de toute part et autres.

Honnêtement, je pense que nous formions un des groupes qui s'entendait le mieux. Il y a certes eu quelques tensions au début du projet, mais elles n'étaient pas d'une très grande ampleur.

De mon point de vue, nous nous sommes bien entendus, voire très bien entendus en très peu de temps. J'ai beaucoup aimé travaillé avec ce groupe et je pense que je n'aurais pas tenu dans un autre. On voyait que tout le monde tenait ce projet à cœur et qu'il était hors de question de l'abandonner. Je pense aussi que si l'un de nous n'avait pas tenu aussi

bien mentalement que dans la réalisation des tâches, les autres auraient fini par abandonner aussi.

Dans un second temps, mon avis sur la réalisation du projet.

Pour commencer, je pense que la répartition des tâches a été assez équitable : tout le monde a rempli ses tâches, avec plus ou moins de difficultés mais qu'est-ce qu'un projet sans difficulté ? En ce qui concerne la réalisation de mes tâches, j'ai bien aimé la conception des niveaux. Je pense que je n'aurais pas pu avoir de meilleures missions. J'ai pu laisser libre cours à mon imagination même si les manques d'inspirations et la pose de tous les colliders m'ont quelques fois déprimée. Assez pour prendre une journée entière de repos.

Les scripts que j'ai dû créer n'étaient pas compliqués à comprendre alors que c'était la partie que je redoutais le plus. C'est pourquoi je compatis avec mes camarades qui ont dû se confronter au débogage. (Tu as ton mon soutien Hengrui.)

En ce qui concerne mon organisation, pour être totalement transparente avec mes camarades, je l'ai volontairement laissée catastrophique... Il faut dire que le projet a été une grande charge mentale pour moi, une grande source de stress. Il fallait suivre le rythme pour y arriver, le rythme des soutenances, le rythme des cours aussi. J'avais conscience de mon organisation inexistante mais j'ai préféré fuir.

Dans tous les cas, je savais que j'aurais rempli mes tâches à temps puisqu'il le fallait. Je repoussais juste l'échéance. On va dire que je n'ai pas eu le courage de me discipliner tout au long de l'année. C'est aussi pour cette raison que je pense que j'aurais abandonné sans ces partenaires.

Mais je pense aussi que c'est parce que j'avais ces partenaires que je me suis autant permis de fuir. Malgré tout je ne pense pas que j'aurais pu mieux faire étant donné la charge de travail qu'est ce projet en plus de celle que nous donnent nos cours.

En ce qui concerne l'avenir, si on me demande de rejoindre un projet jeu vidéo pour un rendu scolaire, je pense que je refuserais : un an n'est pas suffisant pour faire tout ce que nous voulons vraiment faire, surtout si les cours que nous avons ne sont pas en rapport avec ce jeu.

En revanche, rejoindre ce type de projet pour un projet personnel voire sans limite de temps, je pense que j'accepterais."

Hengrui :

"Etant fan des jeux vidéo depuis ma jeunesse, j'ai toujours eu un rêve de pouvoir créer de A à Z un jeu vidéo de mes propres mains.

Depuis que j'ai entendu parler de ce projet, j'ai tout de suite eu un sentiment de joie et de hâte, parce que j'allais enfin pouvoir réaliser mon propre jeu vidéo. Et lorsque j'ai su que ce projet allait se faire en groupe, j'ai eu un sentiment de soulagement mais aussi de stress. Soulagement, parce que je sais que réaliser un jeu tout seul est une tâche difficile, encore plus pour un débutant. De stress, parce que je ne savais pas si mes coéquipiers avaient les mêmes préférences que moi, j'avais peur de faire un jeu que je déteste.

Cependant, ce sentiment de stress a disparu lors de notre première discussion, où nous avons réussi à nous mettre d'accord sur un jeu qui plaisait à tout le monde, c'était le premier pas crucial qui rendait le projet amusant.

Une grosse majorité du temps, la réalisation du projet était vraiment amusante, le fait d'apprendre le fonctionnement des mouvements des personnages, et peu à peu comprendre comment fonctionne les jeux, etc...

Cependant, il reste une petite partie non-négligeable que sont les corrections de bugs qui sont vraiment horribles. On doit y passer beaucoup, mais vraiment beaucoup dessus, mais lorsqu'on réussit à les corriger, le sentiment de satisfaction est aussi incroyable.

Finalement, je trouve que ce projet est une expérience vraiment sympa, qui m'a permis de mieux comprendre la réalisation d'un jeu vidéo, sa difficulté, mais aussi de comprendre comment fonctionnent les travaux en groupe dans le monde du travail. Le seul regret que j'ai ressenti durant la réalisation de ce projet est mon organisation qui était catastrophique, je n'ai pas réussi à maintenir un rythme de travail pour le projet durant l'année, ce qui rendait mon emploi du temps chargée par moment, et qui ralentissait l'avancement dans le projet, et c'est aussi mon plus gros regret concernant le projet, parce que j'aurai pu faire beaucoup mieux."

Nael :

"Concernant mon ressenti à propos du projet, je dirais tout d'abord que cette année n'a pas été de tout repos.

En effet, je viens de loin, toujours en France, mais de l'autre côté de l'océan Atlantique, plus précisément de Guyane française. J'ai été scolarisé là-bas durant les années qui ont précédé mon arrivée en France hexagonale. Ce changement de cadre a impliqué une période d'adaptation, car j'ai dû m'acclimater à la météo de la France, qui est très différente de celle de ma région d'origine. Les températures, le climat, et même les saisons sont des éléments qui ont nécessité un temps d'adaptation pour que je puisse me sentir à l'aise.

De plus, la gestion de mon emploi du temps n'a pas été facile. J'ai dû passer par plusieurs phases de planification afin de pouvoir délibérément consacrer du temps au projet de fin d'année.

Au début, il était difficile de jongler entre mes études, mes obligations personnelles et le travail sur le projet.

Cependant, au fur et à mesure que l'année avançait, j'ai réussi à instaurer des méthodes de planification plus efficaces. Cela m'a permis de respecter la plupart des délais que je m'étais fixés, ce qui a contribué à faire avancer le projet plutôt qu'à le retarder.

En ce qui concerne mon ressenti vis-à-vis de l'équipe, dès le début, j'ai eu le sentiment que, malgré le fait que nous ne nous connaissions pas beaucoup, il y avait un potentiel pour une bonne entente. J'étais convaincu que nous pouvions avancer de manière méthodique et ordonnée pour créer un jeu de qualité. Chacun d'entre nous a pu apporter sa propre pierre à l'édifice, enrichissant ainsi le projet de fin d'année.

Pour ma part, je suis fier du chemin que nous avons parcouru ensemble. Chaque membre de l'équipe a eu l'opportunité de proposer des idées qui lui tenaient à cœur, ce qui a permis d'enrichir notre travail collectif.

Franchement, je pense que le groupe que nous formons est assez complet et que nous avons su tirer parti de nos diversités pour créer quelque chose de vraiment intéressant.”

Ressenti global du groupe :

Au départ, nous étions excités de commencer ce projet. Il s'agit quand même d'un projet jeu vidéo. En tant que passionnés de jeu vidéo, qui n'a jamais rêvé de pouvoir créer son propre jeu vidéo ? Les idées se sont rapidement enchaînées, et même si aucun d'entre nous ne savait comment réaliser nos idées concrètement nous étions convaincus que tout allait bien se passer. Il existe beaucoup de tutoriels sur YouTube après tout.

Au fil du temps, nous nous sommes rendu compte de l'ampleur qu'est de créer un jeu vidéo de toutes pièces. En plus de quoi les tutoriels sont longs, ne font pas forcément ce que l'on souhaite, ne sont pas exactement comme le Unity que nous avons sous les yeux : les tutoriels se font souvent sur des anciennes versions de Unity. Les problèmes se sont vite accumulés : problème d'organisation qui a engendré des périodes de précipitation et du manque de temps, problème lié au partage du projet, des conflits d'accès sur gitHub, problèmes de fusion du projet, problèmes de compilation et beaucoup de soirs à déboguer les codes, d'après-midis entiers à ajouter des fonctionnalités...

Tous ces problèmes nous ont énormément pesés et stressés : le projet est devenu une lourde charge mentale et de travail. C'est un projet qui nécessite du temps et beaucoup d'énergie, choses que nous n'avons pas réussi à garder longtemps. Nous savions que le projet jeu vidéo est la grande épreuve de la première année à l'EPITA, mais toutes les préparations mentales ne sont rien comparées à l'expérience que le vécu nous apporte.

Pour résumer, le projet n'a pas été une simple affaire. Il nous a fait voir de toutes les couleurs. Il a été une source de stress mais également une source de plaisir, d'amusement. Il nous a confrontés à divers obstacles, aussi bien personnellement que pour le groupe. Il a été une source de frustration aussi : nous n'avions pas pu garder toutes nos idées d'origine par manque de temps.

Après toutes ces heures à créer "Objectif Lune!", nous pouvons dire que chacun de nous y a laissé une partie de son âme. Oui. Nous pouvons dire que c'était dur. Assez dur pour en rire nerveusement à chaque fois qu'on nous demandait "Comment se passe votre projet?", assez dur pour avoir dit chacun au moins vingt fois dans la semaine "J'en peux plus du projet, plus jamais je l'ouvre", "Je veux plus voir Unity de ma vie s'il vous plaît...".

Malgré tout, les rires nerveux se transformèrent peu à peu en rire sincère. Malgré tout, "Objectif Lune!" reste un bon projet. Au moins, nous pourrions dire aux prochaines générations que nous l'avons fait, que c'était dur, mais que c'est possible. Le tout, c'est de ne pas succomber au stress. Nous pourrions leur dire que c'est normal d'être frustré de ne pas pouvoir réaliser toutes les fonctionnalités prévues étant donné l'environnement dense de

l'EPITA, mais surtout que c'est possible.

Pour terminer le ressenti global du groupe, “Objectif Lune!” nous a apporté autant de peines que de plaisirs, autant de frustrations que de satisfactions.

7 Conclusion

Pour conclure, nous sommes fiers d'avoir pu réaliser “Objectif Lune!” en moins d'un an, fiers d'avoir pu finir ce premier et long projet et par la même occasion la première année au sein de l'EPITA. Le jeu est disponible au même titre qu'un vrai jeu conçu par des professionnels en plus d'être assez correct. En tant qu'entreprise, l'objectif de Dev&Co, celui de montrer au monde qu'il est possible d'arriver à faire de grande chose à partir de rien, est atteint. Nous sommes fiers non seulement en tant qu'étudiants de l'EPITA, mais aussi en tant que membre du groupe Dev&Co.

8 Source

Graphisme : Coline EAP, étudiante en école d'animation

Contact :

- Instagram : meimeii.draws
- LinkedIn : Coline EAP

Sons : Alexandra TOURE

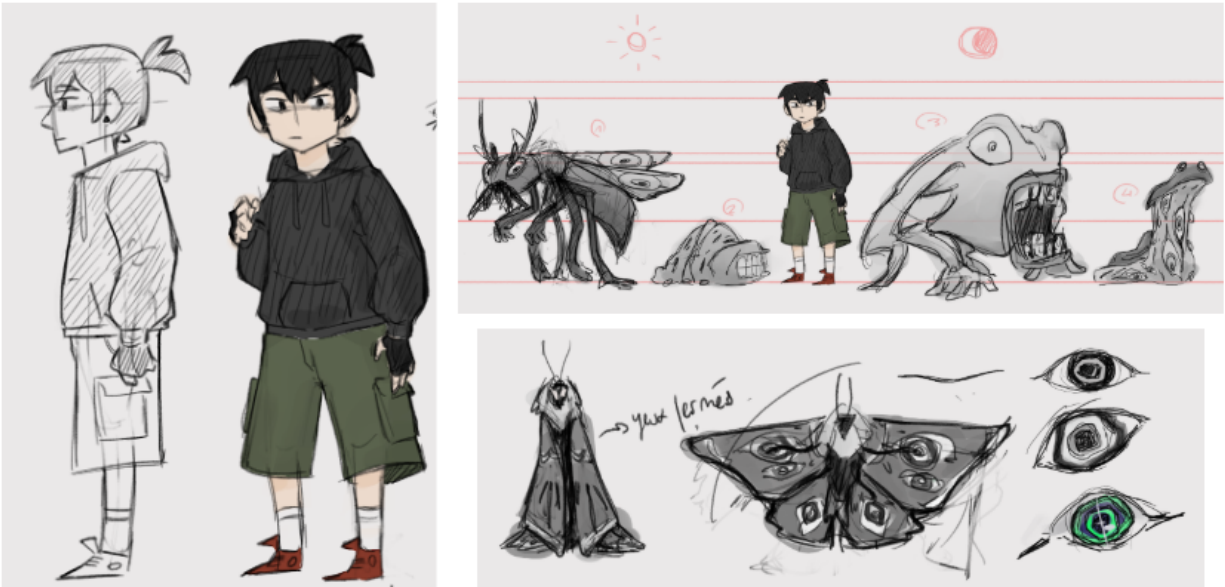
Contact :

- toure.alexandra@yahoo.com
- 0652792744

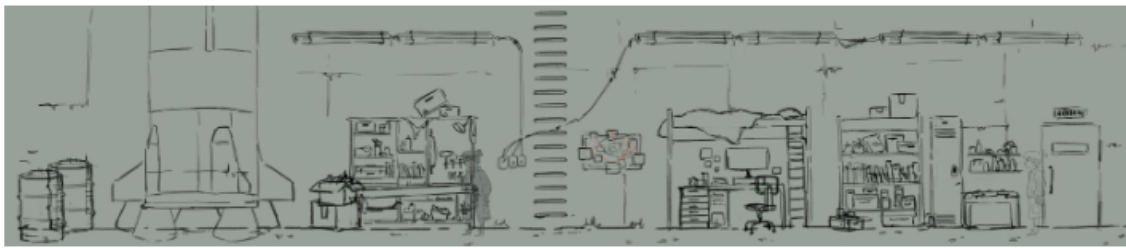
Plateforme : [https ://anokolisa.itch.io/moon-graveyard](https://anokolisa.itch.io/moon-graveyard)

9 Annexes

Évolution des dessins de Sam, le personnage principal et des monstres :



Evolution des dessins du bunker :



Evolution de la jaquette :



Dessins composant le décor des niveaux jour :



Dessins composant le décor des niveaux nuit :

